KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG - UIT

BÁO CÁO THỰC HÀNH

HỌC MÁY TĂNG CƯỜNG CHO CÁC HỆ THỐNG MẠNG

Lab 2

LÀM QUEN VỚI THUẬT TOÁN Q-LEARNING

GVHD: **Phan Trung Phát**

Nhóm: **TH1.01**

Họ và tên	MSSV
Lê Trung Kiên	23520797
Nguyễn Tiến Phát	23521147
Huỳnh Lâm Tuấn Phong	23521162

ĐÁNH GIÁ KHÁC (*):

Nội dung	Kết quả
Tổng thời gian thực hiện bài thực hành trung bình (1)	3 Ngày
Link Video thực hiện (2) *(nếu có)*	
Ý kiến (3) *(nếu có)* + Khó khăn + Đề xuất ...	
Điểm tự đánh giá (4)	10/10

(*): phần (1) và (4) bắt buộc thực hiện.

BẢNG PHÂN CHIA CÔNG VIỆC:

Thành viên	Nội dung thực hiện
Lê Trung Kiên	**Câu 1:** Hoàn thiện bài tập FrozenLake Environment. **Câu 5:** Hoàn thiện Load Balancing Environment Simulation. Custom Gym environment for load balancing.
Nguyễn Tiến Phát	**Câu 5:** Hiện thực chính sách Round Robin. **Câu 6:** Thực hiện chạy môi trường trong 100 Episode **Câu 7:** Viết hàm tổng hợp metrics, vẽ biểu đồ.
Huỳnh Lâm Tuấn Phong	**Câu 2:** Điều chỉnh lại VacuumCleaner Environment, thực hiện Q-learning **Câu 3:** Thực hiện chạy môi trường trong 100 Episode.

Phần bên dưới là báo cáo chi tiết của nhóm/cá nhân thực hiện.

WARNING

LƯU HÀNH NỘI BỘ

© Phan Trung Phát - phatpt@uit.edu.vn

Câu hỏi 1: Đối với FrozenLake Environment, hoàn thành đoạn mã định nghĩa Q-learning còn thiếu trong file Lab2.ipynb, Phần 1. Đồng thời, thêm 1 lựa chọn để thực hiện chính sách Random trong hàm "choose_action" và thực hiện chạy với cùng episode với optimal policy được chọn ra từ thuật toán Q-learning. Lưu trữ lại Reward qua từng episode, vẽ biểu đồ chồng giữa chính sách dựa trên Q-learning và Random. Từ đó, bạn có đánh giá gì đối với từng loại policy ở trên, giải thích, minh chứng.4

1.1. Ý tưởng 4

1.2. Logic code 4

1.2.1 Khởi tạo agent và không gian lưu trữ 4

1.2.2 Cập nhật giá trị Q 4

1.2.3 Chính sách lựa chọn hành động 5

1.2.4 Chính sách tối ưu 6

1.2.5 Huấn luyện và đánh giá 6

1.3. Kết quả: 7

1.4. Nhận xét: 8

Câu 2: Đối với VacuumCleaner Environment, dựa trên các đoạn mã được cung cấp tại câu 1 với FrozenLake Environment, chỉnh sửa lại môi trường để có khả năng tích hợp với Q-learning và tiến hành hiện thực Q-learning trong file Lab2.ipynb, Phần 2. Hiện thực chính sách Round Robin và Priority-based tương tự tại bài thực hành số 1 trên môi trường đã chỉnh sửa. 8

2.1 Khởi tạo môi trường 8

2.1.1. Khởi tạo các tham số môi trường 9

2.2.2. Không gian hành động (Action Space) 9

2.2.3. Không gian trạng thái (Observation Space) 9

2.2 Trạng thái môi trường 10

2.2.1. Hàm _get_obs() – Trả về trạng thái môi trường 10

2.2.2. Hàm encode_state() – Mã hóa trạng thái 10

2.3 Các lựa chọn hành động của Robot và Reward nhận được 12

2.4 Q-learning: 14

2.4.1 Chính sách lựa chọn hành động 14

2.4.2 Huấn luyện Agent 14

2.4.3 Chính sách tối ưu và test agent 15

2.4.4 Kết quả thực thi 17

2.5 Round Robin: 17

2.6 Priority-based: 19

Câu 3: Sau khi huấn luyện với Q-learning, thực hiện chạy môi trường với chính sách dựa trên Q-learning, Round Robin và Priority-based trong 100 episode. Viết hàm để tổng hợp Reward nhận được dưới dạng file .csv, vẽ biểu đồ thể hiện Reward đạt được theo thời gian. 20

3.1 Ý tưởng: 20

3.2 Logic code: 20

3.2.1 Tiến hành chạy 100 episode với chính sách Q-learning, Round Robin và Priority-based20

3.2.2 Lưu metric vời file .csv 22

3.3.3 Vẽ biểu đồ so sánh của 3 chính sách Q-learning, Round Robin và Priority-based 23
3.3.4 Kết quả 24
Câu 4. Thực hiện huấn luyện lại Q-learning với 3 trường hợp, tương ứng với 3 trọng số (Learning Rate, Discount Factor, Exploration Rate): 25
Hãy đánh giá về tốc độ hội tụ trong từng trường hợp, Reward đạt được. Từ đó, bạn có đánh giá gì trên 3 nhóm trọng số trên? 25
Câu hỏi 5: Đối với LoadBalancing Environment, chỉnh sửa lại môi trường để có khả năng tích hợp với Q-learning và tiến hành hiện thực Q-learning trong file Lab2.ipynb, Phần 3. Hiện thực chính sách Round Robin tương tự bài thực hành số 1 trên môi trường đã chỉnh sửa 29
5.1 Ý tưởng: 29
5.2 Logic code: 29
5.2.1. Class Task 29
5.2.2 Class Server 29
5.2.3 Class LoadBalancingEnv 31
Câu hỏi 6: Sau khi huấn luyện với Q-Learning, thực hiện chạy môi trường với chính sách dựa trên Q-learning, Round Robin trong 100 episode. Viết hàm lưu trữ các metrics vào file .csv, các metrics cần lưu bao gồm: tổng thời gian chạy, tổng tác vụ đã tạo ra, số tác vụ được tiếp nhận để xử lý, số lượng tác vụ bị bỏ qua, số lượng tác vụ đã hoàn thành, phần thưởng. 35
6.1 Ý tưởng 35
6.2. Logic code 35
6.2.1. Huấn luyện agent trước khi đánh giá 35
6.2.2. Khởi tạo môi trường và gọi hàm huấn luyện 36
6.2.3 Chạy đánh giá 2 chính sách trên 100 episode 37
6.2.4. Lưu metrics vào file .csv 38
6.2.5. Hàm tổng hợp metrics 39
6.3. Kết quả 40
Câu hỏi 7: Từ các số liệu thu thập được, viết hàm tính để tổng hợp các metrics và điền vào bảng so sánh bên dưới. Vẽ biểu đồ thể hiện sự biến thiên của từng policy theo thời gian với 3 metrics: reward, drop, completion. 41
7.1. Ý tưởng 41
7.2 Logic Code 42
7.2.1. Hàm tổng hợp metrics 42
7.2.2. Chuẩn bị dữ liệu tổng hợp cho bảng so sánh 42
7.2.3. Điền bảng so sánh hiệu năng 43
7.2.4. Vẽ biểu đồ thể hiện sự biến thiên của từng policy theo thời gian 44
7.3 Kết quả thực thi 45
7.4 Nhận xét 47
YÊU CẦU CHUNG 48

Câu hỏi 1: Đối với FrozenLake Environment, hoàn thành đoạn mã định nghĩa Q-learning còn thiếu trong file Lab2.ipynb, Phần 1. Đồng thời, thêm 1 lựa chọn để thực hiện chính sách Random trong hàm "choose_action" và thực hiện chạy với cùng episode với optimal policy được chọn ra từ thuật toán Q-learning. Lưu trữ lại Reward qua từng episode, vẽ biểu đồ chồng giữa chính sách dựa trên Q-learning và Random. Từ đó, bạn có đánh giá gì đối với từng loại policy ở trên, giải thích, minh chứng.

Gợi ý: sử dụng thư viện Matplotlib

1.1. Ý tưởng

Bài toán FrozenLake giống với bài toán VacuumCleaner ở lab1, nhưng có nhiều vật cản hơn. Để giải quyết bài này, nhóm dùng giải thuật Q-learning để huấn luyện agent, cho agent liên tục tương tác với môi trường, sau đó lấy phần thưởng nhận được để cập nhật bảng Q. Bảng này là bảng lưu trữ các giá trị kì vọng của việc thực hiện hành động a tại trạng thái s. Sau khi huấn luyện xong(có đầy đủ bảng Q), ta cho Agent tra bảng và chọn các hành động có điểm số cao nhất để về đích.

Ta so sánh 2 chiến lược Q-learning và Random cùng môi trường này, lưu trữ reward qua từng episode và vẽ biểu đồ chồng để có cái nhìn trực quan giữa 2 chính sách

1.2. Logic code

1.2.1 Khởi tạo agent và không gian lưu trữ

Minh chứng:

```
class QLearningAgent:
    # Agent stores Q-values in a dictionary keyed by (state, action).
    # If a (state, action) pair has never been seen, its Q-value is treated as 0.
    def __init__(self, env, alpha=0.1, gamma=0.9):
        self.env = env
        self.alpha = alpha  # Learning rate: how much to update old Q-values
        self.gamma = gamma  # Discount factor: importance of future rewards
        self.q_table = {}   # Sparse Q-table: {(state, action): q_value}
```

Hình 1. Khởi tạo agent và không gian lưu trữ.

Giải thích:

Agent được khởi tạo với 3 tham số:

- alpha: mặc định là 0.1, thể hiện tốc độ học của agent
- gamma: mặc định là 0.9, thể hiện hệ số chiết khấu. Nếu hệ số chiết khấu gần với 1 thì agent thể hiện sự ưu tiên phần thưởng trong tương lai xa. Nếu hệ số chiết khấu gần với 0 thì agent thể hiện sự ưu tiên ở tương lai gần
- q_table: Bảng Q lưu trữ các giá trị Q(s, a)

1.2.2 Cập nhật giá trị Q

Minh chứng:

```python
def update_q_value(self, state, action, reward, next_state):
    # Q-learning update rule:
    #   Q(s,a) <- (1 - alpha)*Q(s,a) + alpha*(reward + gamma*max_a' Q(s',a'))
    # 1) Estimate best possible future value at next_state
    best_next_q = max(
        self.get_q_value(next_state, a) for a in range(self.env.action_space.n)
    )

    # 2) Current estimate for (state, action)
    current_q = self.get_q_value(state, action)

    # 3) Temporal-difference target
    # Reward for current step + discounted best future value
    target_q = reward + self.gamma*best_next_q

    # 4) Blend old estimate with new target
    # New Q-value is a weighted average of old and new information
    new_q = (1-self.alpha)*current_q + self.alpha*target_q

    # 5) Save updated Q-value
    self.q_table[(state, action)] = new_q
```

Hình 2. Cập nhật giá trị của Q.

Giải thích:

Tại mỗi bước đi, ta cập nhật Q theo công thức:

Q(s,a) <- (1 - alpha)*Q(s,a) + alpha*(reward + gamma*max_a' Q(s',a'))

1.2.3 Chính sách lựa chọn hành động

Minh chứng:

```python
def choose_action(self, state, epsilon=0.1, option="q_learning"):
    if option == "random":
        return self.env.action_space.sample()
    # Epsilon-greedy action selection:
    # - With probability epsilon: explore (random action)
    # - Otherwise: exploit (choose action with highest Q-value)

    # 1) Exploration: random action with probability epsilon
    if random.random() < epsilon:
        return self.env.action_space.sample()

    # 2) Exploitation
    # Compute Q-values for all actions in current state
    q_values = [self.get_q_value(state, a) for a in range(self.env.action_space.n)]
    max_q = max(q_values)

    # Tie-breaking: if multiple actions share max Q, choose randomly among them
    best_actions = [
        a for a in range(self.env.action_space.n)
        if q_values[a] == max_q
    ]
    return random.choice(best_actions)
```

Hình 3. Chính sách lựa chọn các hành động.

Giải thích:

+ Theo yêu cầu đề bài, ta cần so sánh giữa chính sách Q-learning và Random, nếu "option" truyền vào là "random", ta sẽ cho agent đó thực hiện random hành động.

+ Khai báo epsilon = 0.1(mặc định), epsilon là xác suất mà agent sẽ chọn hành động khám phá hay khai thác. Ta dùng hàm random() để tạo ra giá trị ngẫu nhiên trong khoảng [0, 1). Nếu giá trị random < 0.1, ta sẽ cho agent khám phá, ngược lại agent sẽ khai thác(chọn hành động tốt nhất).

+ Agent sẽ dựa vào các q_values đã tính được trong bảng Q để tìm ra action tốt nhất tại trạng thái đó.

+ Nếu có nhiều hành động cùng cho điểm cao nhất, ta chọn random giữa chúng.

1.2.4 Chính sách tối ưu

Minh chứng:

```python
def choose_greedy_action(self, state):
    # Pure greedy policy (epsilon=0):
    # Always choose one of the best actions by Q-value.
    q_values = [self.get_q_value(state, a) for a in range(self.env.action_space.n)]
    max_q = max(q_values)
    best_actions = [a for a, q in enumerate(q_values) if q == max_q]
    return random.choice(best_actions)
```

\

Giải thích:

Sau khi đã có bảng Q, agent sẽ dùng chính sách tối ưu dựa trên bảng Q để đi về đích. Ở đây, chỉ có hành động khai thác, dựa trên hành động đạt được kết quả tốt nhất

1.2.5 Huấn luyện và đánh giá

Minh chứng:

```python
print("1. ĐANG HUẤN LUYỆN (TRAINING) AGENT ... ")
# ==========================================
# PHẦN 1: HUẤN LUYỆN (Bắt buộc phải có)
# ==========================================
for _ in range(num_episodes):
    state, _ = env.reset()
    done = False
    while not done:
        # Chọn hành động có exploration (epsilon=0.1)
        action = agent.choose_action(state, epsilon=0.1)

        # Thực thi
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        # Cập nhật bảng Q-Table (Agent HỌC ở bước này)
        agent.update_q_value(state, action, reward, next_state)

        state = next_state
```

Hình 4. Huấn luyện Q-learning agent

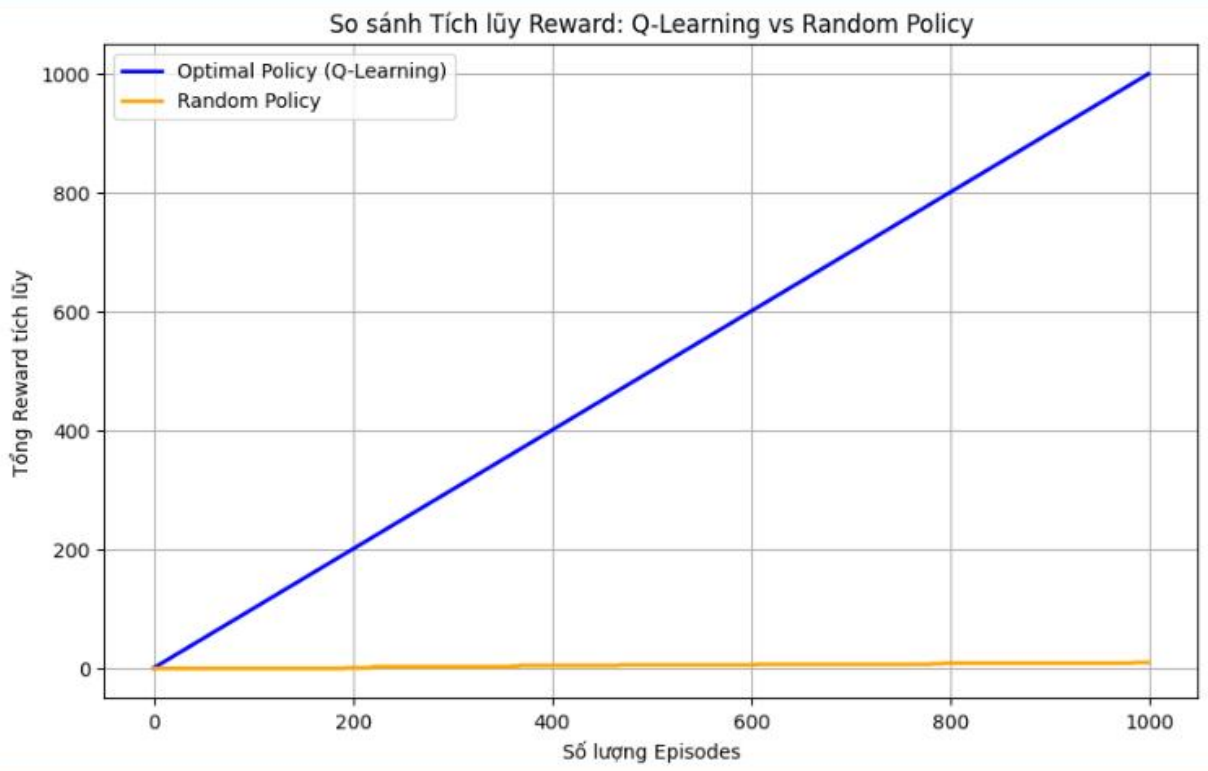
```python
print("Huấn luyện xong! Bắt đầu đánh giá ... ")

q_learning_rewards = []
random_rewards = []

# Optimal Policy(Q-Learning)
for _ in range(num_episodes):
    # Reset environment at start of each episode
    state, _ = env.reset()
    done = False
    episode_reward = 0
    while not done:
        # Dùng hàm choose_greedy_action để chạy tối ưu (không khám phá nữa)
        action = agent.choose_greedy_action(state)
        state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        episode_reward += reward
    q_learning_rewards.append(episode_reward)

# Random Policy
for _ in range(num_episodes):
    state, _ = env.reset()
    done = False
    episode_reward = 0
    while not done:
        # Dùng tuỳ chọn option="random" đã thêm ở trên
        action = agent.choose_action(state, option="random")
        state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        episode_reward += reward
    random_rewards.append(episode_reward)
```

Hình 5. Thực hiện đánh giá sau khi huấn luyện xong

Giải thích:

- Ta huấn luyện agent với giải thuật Q-learning và đánh giá agent với 2 policies là Q-learning và Random.
- **Huấn luyện:** Ta cho agent chạy 1000 episodes và dùng hàm "choose_action" để chọn hành động vừa khai thác vừa khám phá, và hàm "update_value" để liên tục cập nhật giá trị Q sau mỗi step trong mỗi episodes.
- **Đánh giá:** Ta cho agent chỉ thực hiện chính sách tối ưu sau khi xong giai đoạn huấn luyện ở phần Q-learning. Và về phía Random, ta dùng hàm "choose_action" với đối số "random" để agent liên tục thực hiện hành động ngẫu nhiên.
- Trong quá trình đánh giá, ta cập nhật 2 giá trị rewards của mỗi giải thuật trong vòng lặp

1.3. Kết quả:



Hình 6. Biểu đồ thể hiện so sánh tích lũy giữa Q-learning và Random Policy

1.4. Nhận xét:

- Chính sách Random(đường màu cam): Là một đường sát với trục hoành. Lí do là vì môi trường FrozenLake có rất nhiều chướng ngại vật nên việc đi Random như vậy sẽ dẫn đến kết quả thành công gần như bằng 0%.
- Chính sách Q-learning(đường màu xanh): Là một tuyến tính y = x. Thể hiện khả năng thành công gần như 100%.
- Qua đó, ta thấy thuật toán Q-Learning vượt trội hơn so với chính sách Random trong các môi trường phức tạp, nhiều vật cản.

Câu 2: Đối với VacuumCleaner Environment, dựa trên các đoạn mã được cung cấp tại câu 1 với FrozenLake Environment, chỉnh sửa lại môi trường để có khả năng tích hợp với Q-learning và tiến hành hiện thực Q-learning trong file Lab2.ipynb, Phần 2. Hiện thực chính sách Round Robin và Priority-based tương tự tại bài thực hành số 1 trên môi trường đã chỉnh sửa.

2.1 Khởi tạo môi trường

```python
class VacuumCleanerEnv(gym.Env):
    def __init__(self, m=5, n=5, obstacle=(2, 2), obs_mode="encoded"):
        super(VacuumCleanerEnv, self).__init__()
        self.m = m
        self.n = n
        self.obstacle = tuple(obstacle)
        self.obs_mode = obs_mode  # "encoded" hoặc "dict"

        self.action_space = gym.spaces.Discrete(4)

        if obs_mode == "encoded":
            self.observation_space = gym.spaces.Discrete(self.m * self.n * 2)
        else:
            self.observation_space = gym.spaces.Dict({
                "position": gym.spaces.Box(low=0, high=max(self.m, self.n), shape=(2,), dtype=np.int32),
                "dust": gym.spaces.Box(low=0, high=1, shape=(self.m, self.n), dtype=np.int32)
            })

        self.reset()
```

Hình 7. Khởi tạo môi trường

Giải thích:
Trong đoạn code trên, môi trường được xây dựng bằng cách kế thừa từ gym.Env, với các thành phần chính như sau:

2.1.1. Khởi tạo các tham số môi trường

- m, n: kích thước của lưới (grid), mặc định là 5x5
- obstacle: vị trí vật cản trong môi trường, được chuyển thành tuple để đảm bảo tính bất biến
- obs_mode: chế độ biểu diễn trạng thái, gồm 2 dạng:
  - "encoded": mã hóa trạng thái thành số nguyên
  - "dict": biểu diễn trạng thái dưới dạng dictionary

2.2.2. Không gian hành động (Action Space)

self.action_space = gym.spaces.Discrete(4)\

→ Agent có 4 hành động rời rạc, tương ứng với các hướng di chuyển (ví dụ: lên, xuống, trái, phải).

2.2.3. Không gian trạng thái (Observation Space)

Tùy vào obs_mode, không gian trạng thái sẽ được thiết kế khác nhau:

- **Trường hợp "encoded":**
  - self.observation_space = gym.spaces.Discrete(self.m * self.n * 2)
  - Trạng thái được mã hóa thành một số nguyên duy nhất
  - Tổng số trạng thái = số ô trong grid × 2 (do mỗi ô có thể có bụi hoặc không)
- **Trường hợp "dict":**
  - Trạng thái được biểu diễn dưới dạng dictionary gồm 2 thành phần:
  - "Position": Là tọa độ hiện tại của agent
  - "Dust": Là ma trận biểu diễn trạng thái bụi của toàn bộ môi trường

- Kích thước (m, n) với giá trị: 0 (không có bụi), 1 (có bụi)

2.2 Trạng thái môi trường

Minh chứng:

```python
def _get_obs(self):
    if self.obs_mode == "encoded":
        return self.encode_state()
    else:
        return {
            "position": self.position.copy(),
            "dust": self.dust_grid.copy()
        }

def encode_state(self):
    pos = self.position[0] * self.n + self.position[1]
    x, y = self.position
    nearby_dust = 0

    for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
        nx, ny = x+dx, y+dy
        if 0 <= nx < self.m and 0 <= ny < self.n:
            if self.dust_grid[nx, ny] == 1:
                nearby_dust = 1
                break

    return pos * 2 + nearby_dust
```

Hình 8. Hàm trạng thái môi trường

Giải thích:

2.2.1. Hàm _get_obs() – Trả về trạng thái môi trường

Hàm này chịu trách nhiệm cung cấp trạng thái hiện tại của môi trường cho agent, tùy theo chế độ obs_mode:

- Nếu obs_mode == "encoded":

→ Gọi hàm encode_state() để trả về trạng thái dưới dạng số nguyên.

- Ngược lại ("dict"):

→ Trả về một dictionary gồm:

  - "position": vị trí hiện tại của agent (copy để tránh bị thay đổi ngoài ý muốn)
  - "dust": ma trận bụi của toàn bộ môi trường

→ Việc sử dụng .copy() giúp đảm bảo dữ liệu gốc trong môi trường không bị chỉnh sửa ngoài ý muốn trong quá trình agent xử lý.

2.2.2. Hàm encode_state() – Mã hóa trạng thái

Hàm này chuyển trạng thái môi trường từ dạng phức tạp sang một giá trị số nguyên duy nhất để phục vụ cho Q-learning.

Quá trình mã hóa gồm 2 thành phần: Mã hóa vị trí agent và kiểm tra bụi lân cận

Cách mã hóa này giúp giảm đáng kể không gian trạng thái, từ (m × n × 2^(m×n)) xuống còn m × n × 2, giúp Q-learning hội tụ nhanh hơn.

```
def reset(self, *, seed=None, options=None):
    self.position = np.array([0,0], dtype=np.int32)

    self.dust_grid = np.ones((self.m, self.n))

    self.step_count = 0
    self.max_steps = 100

    self.dust_grid[self.obstacle] = 0  # obstacle cell has no dust
    self.total_reward = 0.0
    self.truncated = False
    self.terminated = False


    x, y = self.position
    self.dust_grid[x, y] = 0


    return self._get_obs(), {}
```

Hình 9. Hàm mã hóa trạng thái

Giải thích:

- Hàm reset() được sử dụng để đưa môi trường về trạng thái ban đầu trước mỗi episode, đảm bảo quá trình huấn luyện diễn ra độc lập giữa các lần chạy.

1. Khởi tạo vị trí agent

- Agent luôn bắt đầu tại góc trên bên trái của grid (0, 0)
- Sử dụng numpy array để thuận tiện cho các phép toán sau này

2. Khởi tạo ma trận bụi

- Toàn bộ môi trường ban đầu đều có bụi (1 = dirty)
- Kích thước ma trận là (m, n) tương ứng với kích thước phòng

3. Thiết lập vật cản

- Ô chứa vật cản sẽ không có bụi
- Tránh việc agent nhận reward không hợp lệ khi đi vào vị trí này
- Đồng thời thể hiện rằng đây là ô không thể "làm sạch"

4. Khởi tạo các biến điều khiển episode

- step_count: đếm số bước đã đi
- max_steps: giới hạn số bước trong một episode
- total_reward: tổng reward tích lũy
- terminated: trạng thái kết thúc do đạt mục tiêu
- truncated: trạng thái kết thúc do vượt quá số bước cho phép

Nhận xét:

- Việc reset toàn bộ môi trường về trạng thái "full bụi" giúp bài toán rõ ràng và nhất quán giữa các episode
- Có cơ chế giới hạn số bước (max_steps) giúp tránh vòng lặp vô hạn khi agent học chưa tốt

→ Đây là một bước quan trọng để đảm bảo tính ổn định và chính xác trong quá trình huấn luyện Q-learning.

2.3 Các lựa chọn hành động của Robot và Reward nhận được

```python
def step(self, action):
    # compute candidate new position
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    self.step_count += 1

    if action == 0:   # Up
        candidate = self.position + np.array([-1,0])
    elif action == 1: # Down
        candidate = self.position + np.array([1,0])
    elif action == 2: # Left
        candidate = self.position + np.array([0,-1])
    elif action == 3: # Right
        candidate = self.position + np.array([0,1])
    else:
        candidate = self.position.copy()

    # boundary check
    if (0 <= candidate[0] < self.m) and (0 <= candidate[1] < self.n):
        # obstacle check
        if tuple(candidate) == self.obstacle:
            self.position = candidate.copy()
            reward = -10.0
            self.terminated = True
            self.total_reward += reward

            state = self.encode_state()
            return state, reward, True, False, {}
        else:
            self.position = candidate.copy()
    # else: stay in place
```

Hình 10. Hàm thực thi hành động và tính reward

Giải thích:

Hàm step() là thành phần cốt lõi của môi trường, cho phép agent thực hiện hành động, nhận reward và chuyển sang trạng thái mới.

Dựa vào action, agent sẽ di chuyển theo 4 hướng:

- 0: lên (Up)

- 1: xuống (Down)
- 2: trái (Left)
- 3: phải (Right)

→ Vị trí mới được tính bằng cách cộng vector dịch chuyển vào vị trí hiện tại.

Nếu đi vào ô vật cản:

- Agent vẫn di chuyển vào ô đó
- Nhận reward lớn âm -10.0
- Episode kết thúc (terminated = True)

→ Đây là một cơ chế phạt để tránh agent chọn đường sai

```
x, y = self.position

if self.dust_grid[x, y] == 1:
    reward = 3.0
    self.dust_grid[x, y] = 0
else:
    reward = -0.5

reward -= 0.05
self.total_reward += reward

# check if all cleaned
if np.sum(self.dust_grid) == 0:
    reward += 20.0
    self.terminated = True


if self.step_count >= self.max_steps:
    self.truncated = True
```

Hình 11. Các lựa chọn hành động

Tính reward dựa trên trạng thái ô

- **Nếu ô có bụi:**
  - Nhận reward dương +3.0
  - Đồng thời làm sạch ô đó
- **Nếu ô đã sạch:**
  - Nhận penalty -0.5

→ Tránh việc agent đứng yên hoặc lặp lại hành động không cần thiết

Penalty nhẹ theo bước (-0.05) dùng để khuyến khích agent tìm đường tối ưu (ít bước nhất)

- Nếu toàn bộ môi trường đã sạch:
  - Thưởng lớn +20.0
  - Episode kết thúc

2.4 Q-learning:

2.4.1 Chính sách lựa chọn hành động

```
def robot_policy_qlearning(state, Q, epsilon=0.1):
    if np.random.rand() < epsilon:
        return np.random.randint(Q.shape[1])   # explore
    return np.argmax(Q[state])                 # exploit
```

Hình 12. Chính sách lựa chọn hành động

Giải thích:

Chính sách lựa chọn hành động theo phương pháp ε-greedy được sử dụng trong quá trình huấn luyện Q-learning.

Ý nghĩa của epsilon

- Nếu epsilon lớn:
  - Agent khám phá nhiều hơn
  - Học được đa dạng trạng thái nhưng có thể chậm hội tụ
- Nếu epsilon nhỏ:
  - Agent khai thác nhiều hơn
  - Hội tụ nhanh nhưng dễ bị kẹt ở nghiệm chưa tối ưu

→ Giá trị epsilon = 0.1 là một lựa chọn cân bằng giữa khám phá và khai thác.

2.4.2 Huấn luyện Agent

```
def train_q_learning(env, episodes=500, alpha=0.1, gamma=0.9, epsilon=0.1):
    Q = np.zeros((env.observation_space.n, env.action_space.n))

    rewards_per_episode = []

    for episode in range(episodes):
        state, _ = env.reset()
        terminated = False
        truncated = False

        total_reward = 0

        while not terminated and not truncated:
            action = robot_policy_qlearning(state, Q, epsilon)

            next_state, reward, terminated, truncated, _ = env.step(action)

            # Q update
            Q[state][action] += alpha * (
                reward + gamma * np.max(Q[next_state]) - Q[state][action]
            )

            state = next_state
            total_reward += reward
        rewards_per_episode.append(total_reward)

        epsilon = max(0.05, epsilon * 0.995)

    return Q, rewards_per_episode
```

Hình 13. Huấn luyện Q-Learning agent

Giải thích:

Hàm này thực hiện quá trình huấn luyện agent bằng giải thuật Q-learning thông qua việc tương tác lặp lại với môi trường.

Khởi tạo bảng Q

Kích thước:

- Số dòng: số trạng thái
- Số cột: số hành động

Ban đầu tất cả giá trị Q(s, a) = 0

Chọn hành động (ε-greedy)

Kết hợp giữa:

- Khám phá (random)
- Khai thác (dựa trên Q-table)

Cập nhật giá trị Q (Q-update)

```
Q[state][action] += alpha * (
reward + gamma * np.max(Q[next_state]) - Q[state][action]
)
```

Trong đó:

- alpha: tốc độ học
- gamma: hệ số chiết khấu
- np.max(Q[next_state]): giá trị tốt nhất của trạng thái tiếp theo

→ Giúp agent dần học được giá trị tối ưu cho từng hành động

Giảm epsilon (epsilon decay)

- Giảm dần xác suất khám phá theo thời gian
- Giữ ngưỡng tối thiểu là 0.05

→ Ban đầu khám phá nhiều, sau đó tập trung khai thác

Nhận xét:

- Thuật toán được triển khai đúng chuẩn Q-learning với đầy đủ các thành phần cần thiết
- Có sử dụng epsilon decay, giúp cân bằng giữa exploration và exploitation theo thời gian
- Việc lưu reward theo episode giúp dễ dàng đánh giá quá trình hội tụ

→ Đây là phần cốt lõi quyết định hiệu năng của agent trong toàn bộ bài toán.

2.4.3 Chính sách tối ưu và test agent

```python
def robot_policy_trained(state, Q):
    max_q = np.max(Q[state])
    actions = np.where(Q[state] == max_q)[0]
    return np.random.choice(actions)
    # return np.argmax(Q[state])

def test_agent(env, Q):
    state, _ = env.reset()
    terminated = False
    truncated = False

    env.render()

    while not terminated and not truncated:
        action = robot_policy_trained(state, Q)

        state, reward, terminated, truncated, _ = env.step(action)
        env.render()

    print("Final reward:", env.total_reward + 20.0)  # Add back the bonus for cleaning all dust
```

Hình 14. Chính sách tối ưu và kiểm thử agent

Giải thích:

2.4.3.1 Hàm robot_policy_trained:

Cách hoạt động:

- Lấy giá trị Q lớn nhất tại trạng thái hiện tại
- Tìm tất cả các hành động có cùng giá trị Q lớn nhất
- Chọn ngẫu nhiên một hành động trong số đó

Ý nghĩa:

- Đây là chính sách khai thác hoàn toàn (pure exploitation)
- Không còn yếu tố khám phá (epsilon = 0)
- Việc chọn ngẫu nhiên giữa các action có cùng giá trị giúp:
  - Tránh bias cố định
  - Tăng tính linh hoạt khi nhiều hành động "tốt như nhau"

2.4.3.2 Hàm test_agent:

Hàm này dùng để kiểm tra hiệu năng của agent sau khi đã huấn luyện.

Agent sẽ:

- Chọn hành động theo chính sách đã học
- Thực hiện hành động
- Render lại môi trường sau mỗi bước

Lưu ý:

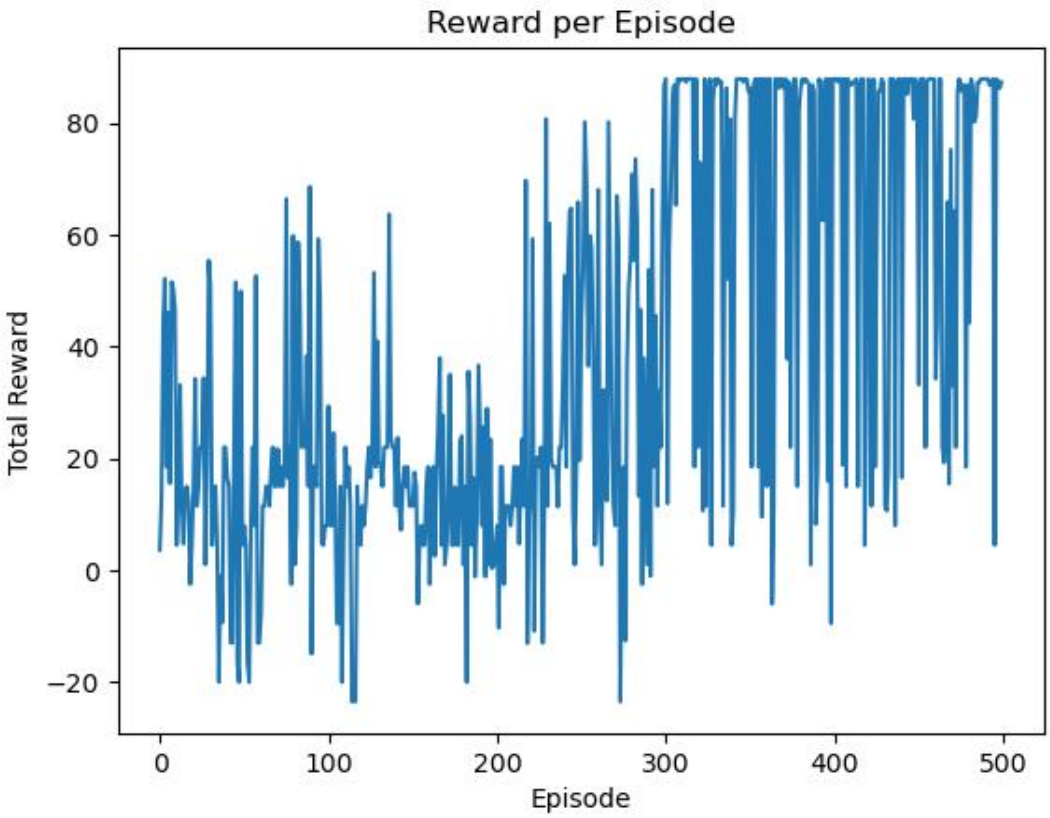
- Có cộng thêm +20.0 là phần thưởng hoàn thành (clean toàn bộ)
- Điều này cho thấy trong quá trình tính total_reward, phần thưởng này chưa được cộng trực tiếp hoặc cần điều chỉnh khi báo cáo

Nhận xét:

- Chính sách sau huấn luyện đã loại bỏ hoàn toàn yếu tố khám phá, phù hợp cho giai đoạn đánh giá
- Việc xử lý trường hợp nhiều action tối ưu là một điểm thiết kế tốt
- Hàm test_agent() cho phép kiểm tra trực quan hành vi của agent, hỗ trợ việc đánh giá kết quả

2.4.4 Kết quả thực thi

```
  #
R

Total reward: 67.85000000000002
Final reward: 87.85000000000002
```



Hình 15. Kết quả sau khi áp dụng Q-learning

2.5 Round Robin:

Minh chứng:

```python
def round_robin_policy(env):
    x, y = 0, 0
    direction = 3  # bắt đầu đi sang phải (3 = right)

    def select_action(obs):
        nonlocal x, y, direction

        m, n = env.m, env.n  # kích thước grid

        # Nếu đang đi sang phải
        if direction == 3:
            if y < n - 1:
                y += 1
                return 3  # right
            else:
                if x < m - 1:
                    x += 1
                    direction = 2  # đổi sang trái
                    return 1  # down

        # Nếu đang đi sang trái
        elif direction == 2:
            if y > 0:
                y -= 1
                return 2  # left
            else:
                if x < m - 1:
                    x += 1
                    direction = 3  # đổi sang phải
                    return 1  # down

        return 0  # fallback (up - ít dùng)

    return select_action
```

Hình 16. Thuật toán Round Robin

Giải thích:

Chính sách Round Robin trong bài toán này được thiết kế để agent di chuyển theo dạng quét toàn bộ grid (zig-zag) thay vì học từ môi trường như Q-learning.

Logic di chuyển dạng zig-zag

- Tiếp tục đi sang phải cho đến khi chạm biên phải
- Khi chạm biên:
  - Di chuyển xuống 1 ô
  - Đổi hướng sang trái

Nhận xét:

- **Ưu điểm:**
  - Đảm bảo đi qua toàn bộ grid → có thể làm sạch toàn bộ môi trường nếu không có vật cản phức tạp
  - Dễ cài đặt, không cần học
- **Nhược điểm:**

- Không thích nghi với môi trường (ví dụ: vật cản)
- Không tối ưu về số bước
- Không sử dụng thông tin trạng thái (obs)

→ Đây là một baseline policy tốt để so sánh với Q-learning, giúp thấy rõ hiệu quả của việc học so với việc di chuyển theo quy luật cố định.

2.6 Priority-based:

```python
def priority_policy():
    def select_action(obs):
        x, y = obs["position"]
        dust = obs["dust"]

        directions = [
            (-1, 0, 0),
            (1, 0, 1),
            (0, -1, 2),
            (0, 1, 3)
        ]

        for dx, dy, action in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < dust.shape[0] and 0 <= ny < dust.shape[1]:
                if dust[nx, ny] == 1:
                    return action

        return np.random.randint(4)

    return select_action
```

✓ 0.0s

Hình 17. Chính sách dựa trên độ ưu tiên

Giải thích:

Chính sách Priority-based là một heuristic policy, trong đó agent ưu tiên làm sạch các ô có bụi ở gần thay vì di chuyển theo quy luật cố định hoặc học từ Q-table.

Nhận xét:

- **Ưu điểm:**
  - Đơn giản, dễ cài đặt
  - Hiệu quả hơn Random vì có định hướng rõ ràng
  - Tận dụng thông tin trạng thái tốt hơn Round Robin
- **Nhược điểm:**
  - Chỉ nhìn được "cục bộ" (xung quanh), không có chiến lược dài hạn
  - Có thể di chuyển không tối ưu nếu bụi ở xa
  - Phụ thuộc vào thứ tự ưu tiên hướng (có thể gây bias)

Câu 3: Sau khi huấn luyện với Q-learning, thực hiện chạy môi trường với chính sách dựa trên Q-learning, Round Robin và Priority-based trong 100 episode. Viết hàm để tổng hợp Reward nhận được dưới dạng file .csv, vẽ biểu đồ thể hiện Reward đạt được theo thời gian.

3.1 Ý tưởng:

Sau khi huấn luyện agent bằng Q-learning, ta muốn so sánh hiệu năng của ba chính sách:

- Q-learning (tối ưu)
- Priority-based (ưu tiên ô bụi gần)
- Round Robin (zig-zag cố định)

Mỗi chính sách sẽ được chạy trong 100 episode, lưu tổng reward nhận được theo từng episode và vẽ biểu đồ so sánh.

3.2 Logic code:

3.2.1 Tiến hành chạy 100 episode với chính sách Q-learning, Round Robin và Priority-based

```python
def run_episode(env, policy_fn, Q=None):
    obs, _ = env.reset()
    terminated = truncated = False
    total_reward = 0

    while not terminated and not truncated:
        if Q is not None:
            action = robot_policy_trained(obs, Q)
        else:
            action = policy_fn(obs)

        obs, reward, terminated, truncated, _ = env.step(action)
        total_reward += reward

    return total_reward
```

Hình 18. Hàm chạy 1 episode

Giải thích:

Hàm run_episode() được sử dụng để chạy một episode hoàn chỉnh của agent trong môi trường, áp dụng cho cả các chính sách như Q-learning, Round Robin và Priority-based.

3.2.1.1 Khởi tạo môi trường

- Gọi reset() để đưa môi trường về trạng thái ban đầu
- terminated: kết thúc do đạt mục tiêu
- truncated: kết thúc do vượt quá số bước
- total_reward: tổng reward tích lũy trong episode

3.2.1.2. Vòng lặp thực thi

Agent sẽ liên tục thực hiện hành động cho đến khi:

- Hoàn thành nhiệm vụ (clean toàn bộ)
- Hoặc vượt quá số bước cho phép

3.2.1.3. Lựa chọn hành động

Nếu có bảng Q:

- Sử dụng chính sách tối ưu đã huấn luyện (robot_policy_trained)

Nếu không:

- Sử dụng policy truyền vào (Round Robin hoặc Priority-based)

3.2.1.4. Thực hiện hành động

- Agent thực hiện action
- Nhận trạng thái mới và reward
- Cộng dồn reward vào total_reward

Nhận xét:

→ Việc tách riêng run_episode() giúp:

- Code rõ ràng, dễ tái sử dụng
- Thuận tiện cho việc chạy nhiều episode để thống kê

```python
def evaluate_policies(episodes=100):
    rr_rewards = []
    pr_rewards = []
    ql_rewards = []

    # Train Q-learning trước
    env_q = VacuumCleanerEnv(m=5, n=5, obstacle=(2, 2), obs_mode="encoded")
    Q, _ = train_q_learning(env_q, episodes=500)

    for ep in range(episodes):
        # ROUND ROBIN
        env = VacuumCleanerEnv(m=5, n=5, obstacle=(2, 2), obs_mode="dict")
        rr_policy = round_robin_policy(env)
        rr_rewards.append(run_episode(env, rr_policy))

        # PRIORITY
        env = VacuumCleanerEnv(m=5, n=5, obstacle=(2, 2), obs_mode="dict")
        pr_policy = priority_policy()
        pr_rewards.append(run_episode(env, pr_policy))

        # Q-LEARNING
        env = VacuumCleanerEnv(m=5, n=5, obstacle=(2, 2), obs_mode="encoded")
        ql_rewards.append(run_episode(env, None, Q))

    return rr_rewards, pr_rewards, ql_rewards
```

Hình 19. Hàm chạy nhiều episode

Hàm evaluate_policies() được sử dụng để **so sánh hiệu năng của ba chính sách**:

- Round Robin
- Priority-based
- Q-learning

thông qua việc chạy nhiều episode và ghi nhận reward tương ứng.

1. Huấn luyện Q-learning trước

- Tạo môi trường với obs_mode="encoded" để phù hợp với Q-table
- Huấn luyện agent trong 500 episode
- Thu được bảng Q tối ưu

→ Việc huấn luyện được thực hiện **một lần duy nhất trước khi đánh giá**

2. Đánh giá Round Robin

- Sử dụng policy zig-zag cố định
- Môi trường ở dạng "dict" để policy có thể truy cập trạng thái trực tiếp
- Lưu reward sau mỗi episode

3. Đánh giá Priority-based

- Agent ưu tiên các ô có bụi gần
- Nếu không có bụi gần → di chuyển ngẫu nhiên
- Reward được lưu lại để so sánh

4. Đánh giá Q-learning

- Sử dụng bảng Q đã huấn luyện
- Agent chỉ thực hiện **chính sách tối ưu (exploit)**
- Không còn yếu tố ngẫu nhiên trong lựa chọn hành động

Nhận xét:

- Hàm đảm bảo mỗi policy được đánh giá trong cùng điều kiện (cùng số episode, cùng môi trường)
- Việc tách riêng môi trường cho từng policy giúp tránh ảnh hưởng lẫn nhau
- Q-learning được huấn luyện trước → đảm bảo so sánh với các policy khác ở trạng thái tối ưu

3.2.2 Lưu metric vời file .csv

```python
import pandas as pd

def save_to_csv(rr, pr, ql, filename="results.csv"):
    df = pd.DataFrame({
        "Episode": list(range(1, len(rr)+1)),
        "RoundRobin": rr,
        "Priority": pr,
        "Qlearning": ql
    })
    df.to_csv(filename, index=False)
```

Hình 20. Hàm để lưu file CSV

Giải thích:

Hàm save_to_csv() được sử dụng để lưu kết quả reward của các chính sách sau khi đánh giá ra file .csv, phục vụ cho việc phân tích và trực quan hóa dữ liệu.

Sử dụng thư viện pandas để tạo bảng dữ liệu

Các cột bao gồm:

- Episode: số thứ tự episode (từ 1 → N)
- RoundRobin: reward của policy Round Robin
- Priority: reward của policy Priority-based
- Qlearning: reward của policy Q-learning

Xuất file CSV

- Ghi dữ liệu ra file .csv
- filename: tên file hoặc đường dẫn lưu file
- index=False: không ghi cột chỉ số (index) của DataFrame vào file

3.3.3 Vẽ biểu đồ so sánh của 3 chính sách Q-learning, Round Robin và Priority-based

```python
import matplotlib.pyplot as plt

def plot_comparison(rr, pr, ql):
    plt.plot(rr, label="Round Robin")
    plt.plot(pr, label="Priority-based")
    plt.plot(ql, label="Q-learning")

    plt.xlabel("Episode")
    plt.ylabel("Reward")
    plt.title("Reward Comparison")
    plt.legend()
    plt.grid()

    plt.show()
```

Hình 21. Hàm vẽ biểu đồ

Giải thích:

Hàm plot_comparison() được sử dụng để trực quan hóa kết quả reward theo từng episode của ba chính sách: Round Robin, Priority-based và Q-learning.

Vẽ các đường biểu diễn

- Mỗi danh sách reward (rr, pr, ql) được biểu diễn thành một đường trên đồ thị
- Label dùng để phân biệt các chính sách trong chú thích (legend)

Nhận xét:

- Biểu đồ giúp so sánh trực quan hiệu năng giữa các chính sách qua từng episode

```
if __name__ == "__main__":
    episodes = 100

    rr, pr, ql = evaluate_policies(episodes)

    save_to_csv(rr, pr, ql, "comparison_rewards.csv")

    plot_comparison(rr, pr, ql)
```

Hình 22. Hàm main

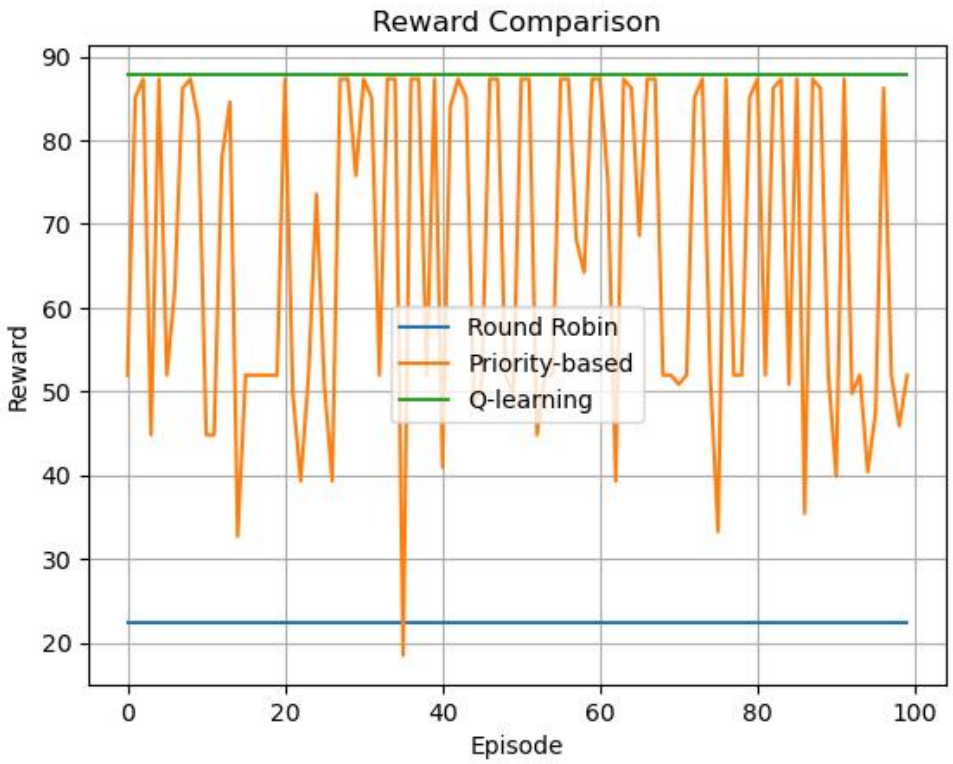
Giải thích:

Đây là phần điều khiển chính của chương trình, thực hiện toàn bộ pipeline từ đánh giá → lưu dữ liệu → trực quan hóa kết quả.

Trong đó:

- evaluate_policies: chạy mỗi policy 100 episode, lưu reward từng episode
- save_to_csv: xuất reward ra file .csv để lưu trữ, dễ phân tích
- plot_comparison: vẽ biểu đồ reward theo thời gian để so sánh trực quan
- Với Q-learning, agent sử dụng robot_policy_trained → chỉ khai thác (exploit) bảng Q đã huấn luyện
- Priority-based và Round Robin sử dụng logic di chuyển heuristic đã định nghĩa ở trên

3.3.4 Kết quả



Hình 23. So sánh phần thưởng đạt được theo thời gian

Nhận xét:

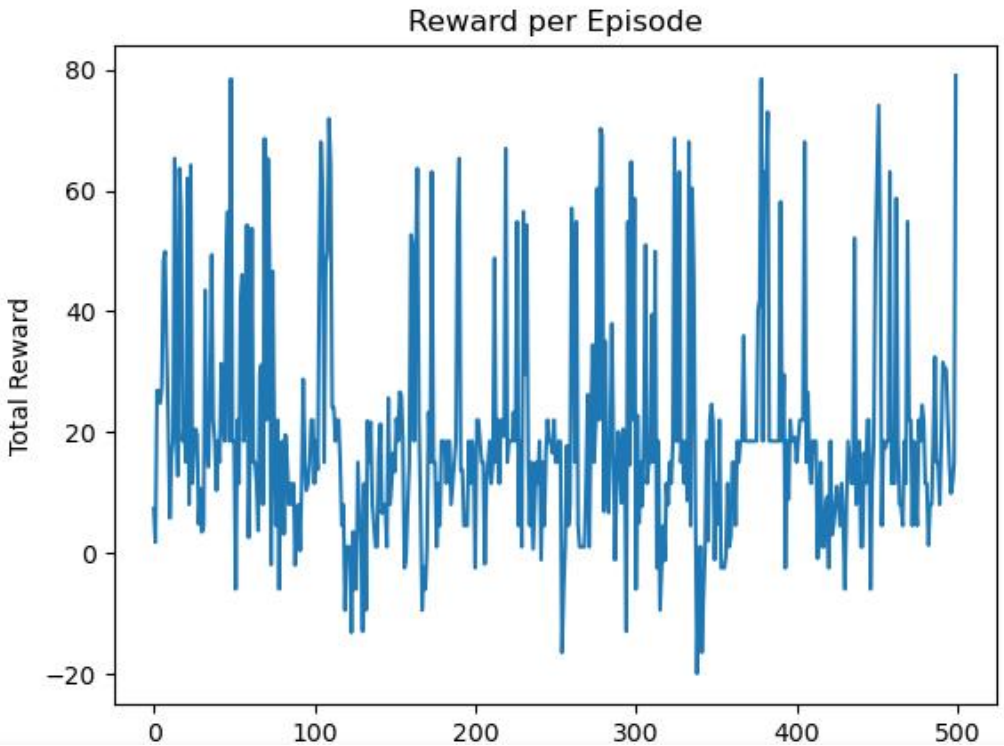
- **Round Robin :** reward ổn định, gần như đường thẳng, nhưng thấp hơn hẳn so với 2 gaiir thuật còn lại
- **Priority-based:** reward trung bình, dao động không ổn định theo từng episode
- **Q-learning:** reward cao, gần như tuyến tính y = x, thể hiện khả năng hoàn thành nhiệm vụ gần như 100%

Câu 4. Thực hiện huấn luyện lại Q-learning với 3 trường hợp, tương ứng với 3 trọng số (Learning Rate, Discount Factor, Exploration Rate):

- **Trường hợp 1: (0.5, 0.2, 0.1)**
- **Trường hợp 2: (0.1, 0.95, 0.2)**
- **Trường hợp 3: (0.1, 0.8, 0.7)**

Hãy đánh giá về tốc độ hội tụ trong từng trường hợp, Reward đạt được. Từ đó, bạn có đánh giá gì trên 3 nhóm trọng số trên?

```
....
..# R
...
...
Total reward: -16.50000000000043
Final reward: 3.4999999999999574
```

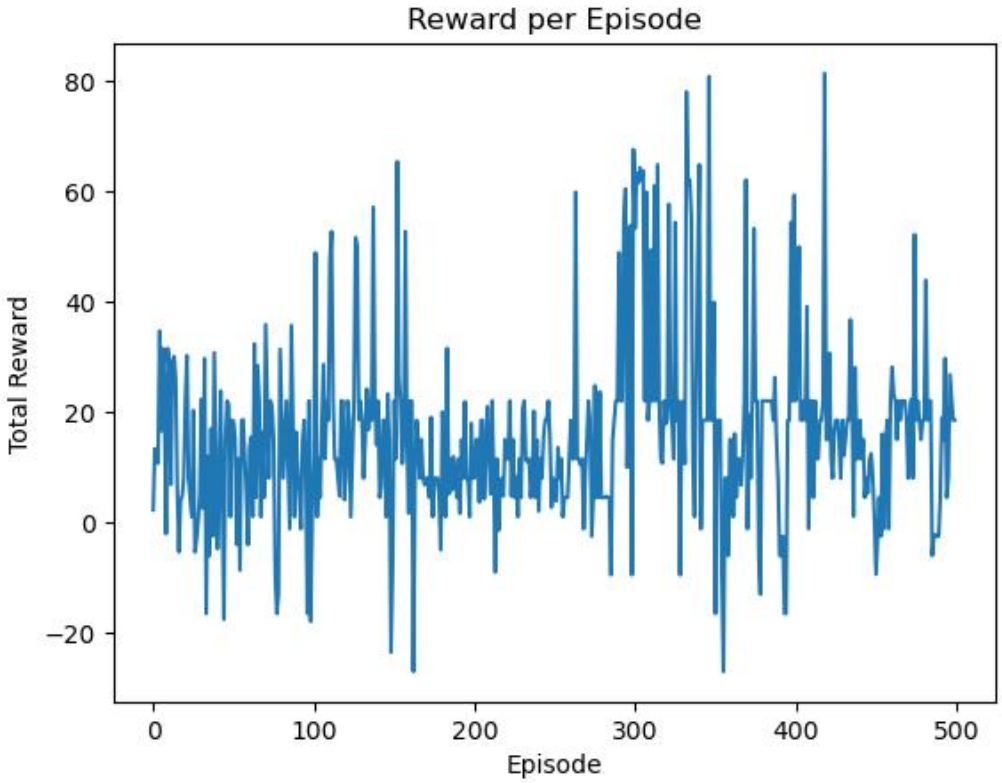


Hình 24. Huấn luyện Q-Learning với trường hợp 1

Trường hợp 1 (0.5, 0.2, 0.1): Tốc độ học cao, tầm nhìn ngắn, ít khám phá

- **Tốc độ hội tụ:** Khá nhanh ở giai đoạn đầu nhưng không ổn định.
  - Với alpha = 0.5, agent cập nhật giá trị Q rất mạnh tay, dẫn đến biểu đồ dao động liên tục (nhiễu cao).
  - Tuy nhiên, vì gamma = 0.2 (tầm nhìn rất ngắn), agent chỉ quan tâm đến phần thưởng tức thì, khiến nó dễ rơi vào việc không hy sinh lợi ích nhỏ trước mắt nhằm đạt được phần thưởng lớn.
- **Reward:** Reward đạt được không cao và không duy trì được mức đỉnh ổn định.

```
   ..
   ..
R #..
   ..
   ..
Total reward: -9.499999999999996
Final reward: 10.500000000000004
```

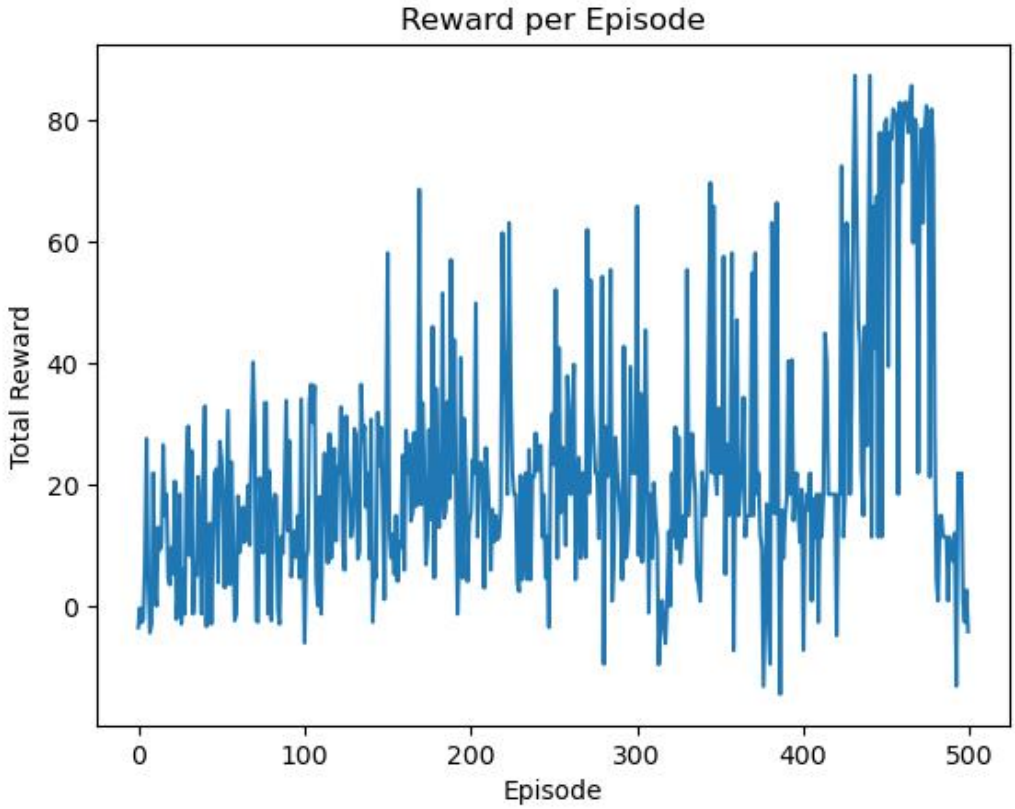


Hình 25. Huấn luyện Q-Learning với trường hợp 2

Trường hợp 2 (0.1, 0.95, 0.2): Tốc độ học chậm, tầm nhìn xa, khám phá vừa phải

- **Tốc độ hội tụ:** Bắt đầu có sự cải thiện rõ rệt so với TH1
  - Agent học một cách cẩn trọng. Điều này giúp đồ thị bớt những cú "nhảy dựng" quá mức và giữ được sự ổn định của những gì đã học trước đó.
  - Biểu đồ cho thấy agent mất nhiều thời gian hơn để bắt đầu hiểu môi trường. Tuy nhiên, nhờ gamma = 0.95 (ưu tiên phần thưởng trong tương lai), agent có xu hướng tìm ra chiến lược tối ưu dài hạn tốt hơn.
- **Reward:** Mức Reward bắt đầu có dấu hiệu tăng dần về phía cuối (khoảng sau episode 300), cho thấy sự cải thiện chiến thuật rõ rệt hơn so với trường hợp 1.

```
....
..#.
....
....R
Total reward: -27.000000000000043
Final reward: -7.000000000000043
```



Hình 26. Huấn luyện Q-Learning với trường hợp 3

Trường hợp 3 (0.1, 0.8, 0.7): Tốc độ học chậm, tầm nhìn trung bình, khám phá mạnh

- **Tốc độ hội tụ:** Rất chậm và biến động cực kỳ lớn.
  - Với epsilon = 0.7, agent dành tới 70% thời gian để hành động ngẫu nhiên (Exploration).
  - Điều này giải thích tại sao biểu đồ nhảy vọt liên tục và có những cú rơi sâu về Reward âm ngay cả ở những episode cuối.
- **Reward:** Dù có những thời điểm đạt Reward rất cao (do khám phá được đường đi tốt), nhưng vì tính ngẫu nhiên quá lớn, agent không "chốt" được chính sách ổn định.

Nhận xét:

Nhóm hiệu quả nhất: Trường hợp 2 là nhóm thông số cân bằng nhất. Trong Reinforcement Learning, một alpha nhỏ giúp việc học bền vững và một gamma lớn giúp agent tối ưu hóa lợi ích lâu dài thay vì chỉ ăn may trước mắt.

Vấn đề của Trường hợp 3: Tỷ lệ khám phá (epsilon) quá cao khiến agent không bao giờ thực sự tận dụng (Exploit) những gì đã học được.

Vấn đề của Trường hợp 1: Việc đặt Learning Rate quá cao làm cho thuật toán khó hội tụ về một giá trị tối ưu mà sẽ "nhảy" quanh điểm đó mãi mãi.

Câu hỏi 5: Đối với LoadBalancing Environment, chỉnh sửa lại môi trường để có khả năng tích hợp với Q-learning và tiến hành hiện thực Q-learning trong file Lab2.ipynb, Phần 3. Hiện thực chính sách Round Robin tương tự bài thực hành số 1 trên môi trường đã chỉnh sửa

5.1 Ý tưởng:

Để áp dụng RL vào việc tìm ra chiến lược điều phối tối ưu, nhóm đã tiến hành xây dựng một môi trường mô phỏng bằng thư viện Gymnasium. Môi trường này mô hình hóa bài toán thành một Quá trình quyết định Markov (MDP), bao gồm 3 thực thể chính:

- **Task:** Đại diện cho các luồng xử liệu hoặc request gửi đến hệ thống
- **Server:** Đại diện cho các node xử lý có tài nguyên giới hạn
- **LoadBalancingEnv**: Đóng vai trò là environment mà ở đây là hệ thống mạng tổng thể, cung cấp không gian quan sát (state) cho tác nhân, tiếp nhận action và trả về reward

5.2 Logic code:

5.2.1. Class Task

Minh chứng:

```
class Task:
    """A task with a unique id and required processing time (in time steps)."""

    def __init__(self, task_id: int, processing_time: int):
        self.task_id = task_id  # Unique identifier for tracking
        self.processing_time = processing_time  # Number of steps needed to finish
```

Hình 27. Hàm khởi tạo cho Class Task

Giải thích:

Trong hệ thống mạng, Task được xem là một gói tin hoặc một luồng yêu cầu. Ta cung cấp cho mỗi Task một "task_id" giúp hệ thống dễ dàng theo dõi vòng đời của nó, từ lúc sinh ra, được chấp nhận, cho đến khi hoàn thành hoặc bị dropped. "processing_time" mô phỏng số bước để hoàn thành.

5.2.2 Class Server

Class Server đóng vai trò là các Worker Node. Mục đích của class này là mô phỏng lại các ràng buộc vật lý của một máy chủ thực tế như: CPU chỉ có thể xử lý một việc tại một thời điểm(current_task), và bộ nhớ đệm có giới hạn(queue_capacity)

5.2.2.1 Xử lý công việc theo thời gian thực (Hàm run_one_step)

Minh chứng:

```python
def run_one_step(self):
    """
    Execute one simulation step on this server.

    Returns:
        completed_task (Task | None): task completed at this step, if any.
    """
    completed_task = None

    # Process one time unit for currently running task
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    if self.current_task is not None:
        self.remaining_time -= 1
        if self.remaining_time == 0:
            completed_task = self.current_task
            self.current_task = None
            self.remaining_time = 0

    # If server becomes idle, immediately pull next task from queue
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    else:
        if len(self.queue) > 0:
            self.current_task = self.queue.popleft()
            self.remaining_time = self.current_task.processing_time

    return completed_task
```

Hình 28. Hàm run_one_step

Giải thích:

Môi trường RL hoạt động theo từng time steps. Hàm này giúp Server đồng bộ hóa với đồng hồ chung của hệ thống. Mỗi khi hàm được gọi, máy chủ sẽ tiêu tốn 1 đơn vị thời gian để giai quyết công việc hiện tại. Khi CPU rảnh rỗi, hệ thống phải tự động kéo công việc tiếp theo từ hàng đợi qua "queue.popleft()" lên để xử lý.

5.2.2.2 Thêm Task cho server xử lý (Hàm add_task)

Minh chứng:

```python
def add_task(self, task: Task) -> bool:
    """
    Try to add a task to this server.

    Rules:
    - If server is idle: start processing immediately.
    - Else if queue has room: enqueue task.
    - Else: reject (drop) task.
    Returns:
        bool: True if accepted, False if dropped.
    """
    # Start immediately if server is free
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    if self.current_task is None:
        self.current_task = task
        self.remaining_time = self.current_task.processing_time
        return True
    # Otherwise enqueue if capacity allows
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    else:
        # Queue not full -> enqueue task
        if len(self.queue) < self.queue_capacity:
            self.queue.append(task)
            return True
        # Queue full -> task dropped
        return False
```

Hình 29. Hàm add_task

Giải thích:

Khi Agent quyết định đẩy 1 Task cho Server bất kì, nó phải kiểm tra sức chứa. Kết quả của việc kiểm tra dẫn đến 3 trường hợp.

- Nếu Server rảnh: Nhận Task ngay
- Nếu Server không rảnh và hàng đợi của Server đó vẫn còn chỗ chứa: Thêm Task vào cuối hàng đợi
- Các trường hợp còn lại: Drop Task

Kết quả trả về là True và False để làm điều kiện cho tính toán thưởng phạt cho Agent.

5.2.3 Class LoadBalancingEnv

Class LoadBalancingEnv là môi trường, là trung tâm của bài toán, nơi quản lý các Server, sinh ra các Task, và là nơi định nghĩa rõ ràng bài toán MDP

5.2.3.1 Khởi tạo không gian MDP (Hàm __init__)

Minh chứng:

```python
def __init__(self, n_servers: int = 3, queue_capacity: int = 2, seed: int = None):
    super().__init__()
    self.n_servers = n_servers
    self.queue_capacity = queue_capacity
    self.rng = random.Random(seed)  # Local random generator for reproducibility
    # Create server pool
    self.servers = [Server(queue_capacity) for _ in range(n_servers)]
    self.time = 0  # Global simulation step
    self.total_reward = 0.0  # Accumulated reward over episode

    # Task tracking
    self.next_task_id = 0
    self.tasks_created = {}     # task_id -> Task
    self.tasks_completed = set()  # IDs of completed tasks
    self.tasks_dropped = set()  # IDs of dropped tasks
    self.tasks_accepted = set()  # IDs of accepted tasks
    # RL spaces
    # Define action space with n_servers
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    self.action_space = gym.spaces.Discrete(self.n_servers)

    # Define observation space as a dict containing server states and global time
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    self.observation_space = gym.spaces.Dict({
        "servers": gym.spaces.Tuple([
            gym.spaces.Dict({
                "remaining_time": gym.spaces.Box(low=0, high=float('inf'), shape=(), dtype=float),
                "queue_length": gym.spaces.Discrete(self.queue_capacity + 1)
            }) for _ in range(self.n_servers)
        ]),
        "time": gym.spaces.Box(low=0, high=float('inf'), shape=(), dtype=float)
    })
```

Giải thích:

Với mọi bài toán RL, ta cần xác định MDP của bài toán đó. Đầu tiên là không gian hành động (Action Space). Mỗi khi 1 Task xuất hiện, quyết định duy nhất mà Load Balancer (Agent) đưa ra là chọn đích đến cho Task đó, được thể hiện qua "self.action_space = gym.spaces.Discrete(self.n_servers). Nếu hệ thống có 3 máy chủ (n_servers = 3), Agent sẽ xuất ra một số nguyên rời rạc 0, 1, 2 tương ứng với việc đẩy Task vào Server 0, Server 1 hoặc Server 2

5.2.3.2 Khôi phục lại hệ thống và mô phỏng lưu lượng (Hàm _new_task và hàm reset)

Minh chứng:

```python
def _new_task(self) -> Task:
    """Create one new incoming task with random processing time [1, 5]."""
    # Create a new task with a unique ID and random processing time between 1 and 5
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    t = Task(self.next_task_id, self.rng.randint(1, 5))
    self.tasks_created[t.task_id] = t
    self.next_task_id += 1
    return t
```

Hình 30. Hàm _new_tasks

```python
def reset(self, *, seed=None, options=None):
    """Reset environment state and all tracking metrics."""
    # Optional reseed to make episode deterministic from this point
    if seed is not None:
        self.rng.seed(seed)

    # Reinitialize server states, time, and rewards
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    self.servers = [Server(self.queue_capacity) for _ in range(self.n_servers)]
    self.time = 0
    self.total_reward = 0.0

    # Clear all task statistics
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    self.next_task_id = 0
    self.tasks_created = {}
    self.tasks_completed = set()
    self.tasks_dropped = set()
    self.tasks_accepted = set()


    # Gymnasium-style reset return: (observation, info)
    return self._get_observation(), {}
```

Hình 31. Hàm reset()

Giải thích:

+ Hàm "_new_task" tạo ra yếu tố ngẫu nhiên của môi trường bằng cách sinh ra các công việc có thời lượng xử lý ngẫu nhiên từ 1 đến 5. Agent không thể biết trước task tiếp theo nặng hay nhẹ, buộc nó phải có chiến lược dự phòng linh hoạt

+ Hàm "reset" giải phóng toàn bộ bộ nhớ, khởi tạo lại các Server trống và xóa sạch các chỉ số đo lường hiệu năng.

5.2.2.3 Đánh giá Hành động và Định hình Hàm thưởng (Hàm step)

Minh chứng:

```python
def step(self, action: int):
    """
    Run one simulation step:
    1) Advance all servers by one time step.
    2) Generate one new task.
    3) Route task to selected server.
    4) Compute reward and return transition tuple.
    """
    reward = 0.0
    # 1) Process running tasks on each server
    # For each server, call run_one_step() to advance processing. If a task completes, add to completed set and give a positive reward (+2.0)
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    for server in self.servers:
        completed_task = server.run_one_step()
        if completed_task is not None:
            self.tasks_completed.add(completed_task.task_id)
            reward += 2.0

    # 2) Generate one new incoming task
    new_task = self._new_task()

    # 3) Route task based on action (selected server index)
    # Try to add the new task to the selected server.
    # If accepted, add to accepted set and give a small reward (+0.5).
    # If dropped, add to dropped set and give a strong penalty (-5.0).
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    if(self.servers[action].add_task(new_task)):
        self.tasks_accepted.add(new_task.task_id)
        reward += 0.5
    else:
        self.tasks_dropped.add(new_task.task_id)
        reward -= 5.0

    # 4) Add congestion penalty proportional to queue sizes
    # To encourage the agent to balance load and avoid long queues, subtract a small penalty (e.g., -0.5) for each task waiting in any server's queue.
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    for server in self.servers:
        reward -= 0.5 * len(server.queue)

    # Update global counters
    self.total_reward += reward
    self.time += 1
    # This environment currently never ends by itself
    terminated = False
    truncated = False
    info = self._get_info()

    return self._get_observation(), reward, terminated, truncated, info
```

Giải thích:

Hàm "step" quy định cách thức hệ thống mạng phản ứng lại quyết định của Agent. Đây là nơi định nghĩa Hàm phần thưởng (Reward Function) - yếu tố then chốt để quyết định việc làm của Agent tốt đến đâu. Nhóm đã thiết kế cấu trúc thưởng phạt gồm 4 logic chính:

+ Duy trì hiệu suất (+2.0): Khuyến khích hệ thống hoàn thành nhiều Task nhất có thể.

+ Khuyến khích tiếp nhận (+0.5): Thưởng nhẹ khi định tuyến thành công một Task vào Server và được chấp nhận(Hàm trả về True).
+ Trừng phạt quá tải (-5.0): Xảy ra khi Agent đẩy Task vào Server đang bận và đầy hàng đợi(Hàm trả về False).

+ Trừng phạt tắc nghẽn cục bộ (-0.5 * số Task đang đợi): Logic này ép Agent phải phân bổ đồng đều ra tất cả các Server, thay vì chỉ dồn hết vào một chỗ cho đến khi đầy.

5.2.2.4 Đo lường hiệu năng mạng (Hàm _get_info):

Minh chứng:

```python
def _get_info(self):
    """Return useful metrics for logging/evaluation."""
    # Compute metrics based on requirements on your lab assignment. Here are some examples:
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    total_created = len(self.tasks_created)
    dropped = len(self.tasks_dropped)
    completed = len(self.tasks_completed)
    accepted = len(self.tasks_accepted)

    # Safe division to avoid divide-by-zero at the beginning
    # Calculate some key performance metrics
    ### YOU NEED TO WRITE YOUR CODE BELOW ###
    # HERE
    if total_created > 0:
        drop_rate = dropped / total_created
    else:
        drop_rate = 0.0
    #...

    return {
        # Some example info fields: time, dropped tasks, dop_rate, etc.
        "time": self.time,
        "total_created tasks": total_created,
        "dropped tasks": dropped,
        "completed tasks": completed,
        "accepted tasks": accepted,
        "drop rate": drop_rate
    }
```

Mục đích:

Ngoài việc tính điểm Reward cho máy học, hệ thống cần cung cấp các chỉ số thực tế(Metrics) để chúng ta đánh giá chất lượng của Agent(Load Balancer). Chỉ số quan trọng nhất ở đây là "drop_rate". Một thuật toán định tuyến tốt là giữ cho "drop_rate" tiệm cận mức 0%. Dữ liệu này sẽ đợc dùng để vẽ biểu đồ so sánh giữa các chiến lược sau quá trình mô phỏng.

Câu hỏi 6: Sau khi huấn luyện với Q-Learning, thực hiện chạy môi trường với chính sách dựa trên Q-learning, Round Robin trong 100 episode. Viết hàm lưu trữ các metrics vào file .csv, các metrics cần lưu bao gồm: tổng thời gian chạy, tổng tác vụ đã tạo ra, số tác vụ được tiếp nhận để xử lý, số lượng tác vụ bị bỏ qua, số lượng tác vụ đã hoàn thành, phần thưởng.

6.1 Ý tưởng

Sau khi hoàn tất giai đoạn huấn luyện, nhóm sử dụng chính sách Q-learning đã học được để chạy đánh giá trên môi trường LoadBalancing. Đồng thời, nhóm triển khai thêm chính sách Round Robin để làm mốc so sánh. Cả hai chính sách đều được chạy trong 100 episode trên cùng môi trường đã chỉnh sửa.

Trong mỗi episode, hệ thống thu thập các chỉ số hiệu năng quan trọng gồm: tổng thời gian chạy, tổng số task được tạo ra, số task được tiếp nhận, số task bị drop, số task đã hoàn thành và tổng reward. Sau đó, các metrics này được lưu vào file .csv để phục vụ cho bước tổng hợp và so sánh ở phần tiếp theo.

6.2. Logic code

6.2.1. Huấn luyện agent trước khi đánh giá

Minh chứng:

```python
def train_q_learning_agent(env, alpha=0.1, gamma=0.9, num_episodes=5000, steps_per_episode=100, epsilon=0.1):
    """Train a Q-learning agent on the load balancing environment."""
    agent = QLearningAgent(env, alpha=alpha, gamma=gamma)

    for episode in range(num_episodes):
        obs, _ = env.reset()
        state = obs

        for step in range(steps_per_episode):
            # Chọn hành động (có khám phá epsilon=0.1)
            action = agent.choose_action(state, epsilon=epsilon)

            # Tương tác
            next_obs, reward, terminated, truncated, _ = env.step(action)
            next_state = next_obs

            # Học và cập nhật Q-Table
            agent.update_q_value(state, action, reward, next_state)
            state = next_state

    return agent
```

✓ 0.0s Python

Hình 32. Huấn luyện Q-Learning Agent

Giải thích:

Nhóm tách riêng phần huấn luyện thành một hàm độc lập để làm rõ hai pha của bài toán: huấn luyện và đánh giá.

Trong hàm này, agent được cho tương tác nhiều lần với môi trường. Ở mỗi bước, agent chọn hành động bằng cơ chế vừa khám phá vừa khai thác thông qua epsilon, sau đó nhận reward từ môi trường và cập nhật lại **Q-table** bằng hàm *update_q_value*.

Cách tổ chức này giúp phần đánh giá phía sau chỉ sử dụng policy đã học được, không còn làm thay đổi **Q-table** nữa. Nhờ đó, kết quả thu thập được phản ánh đúng chất lượng của chính sách sau huấn luyện.

6.2.2. Khởi tạo môi trường và gọi hàm huấn luyện

Minh chứng:

```python
if __name__ == "__main__":
    # Create environment and reset initial state
    env = LoadBalancingEnv(n_servers=3, queue_capacity=2, seed=42)
    num_train_episodes = 5000
    steps_per_episode = 100

    print(f"1. Đang huấn luyện Q-Learning Agent ({num_train_episodes} episodes)...")
    agent = train_q_learning_agent(
        env,
        alpha=0.1,
        gamma=0.9,
        num_episodes=num_train_episodes,
        steps_per_episode=steps_per_episode,
        epsilon=0.1
    )

    print(f"Huấn luyện xong! Agent đã học được {len(agent.q_table)} trạng thái.")
```

Giải thích:

Trong khối main, nhóm tiến hành khởi tạo một môi trường LoadBalancingEnv với 3 servers, sức chứa hàng đợi của mỗi server là 2, và seed ban đầu là 42.

Sau đó, nhóm thiết lập số episode huấn luyện là 5000, số bước (step) tối đa cho mỗi episode huấn luyện là 100, sau đó gọi hàm train_q_learning_agent() đã định nghĩa trước đó để tiến hành huấn luyện mô hình.

Việc đặt phần huấn luyện trong block main giúp chương trình có luồng chạy rõ ràng: đầu tiên là khởi tạo môi trường, tiếp theo là huấn luyện agent, và cuối cùng mới chuyển sang giai đoạn đánh giá.

Sau khi huấn luyện hoàn tất, hệ thống in ra số lượng trạng thái mà agent đã học được trong Q-table, từ đó làm đầu vào cho bước đánh giá ở phía sau.

Việc đặt phần huấn luyện trong block main giúp chương trình có luồng chạy rõ ràng: đầu tiên là khởi tạo môi trường, tiếp theo là huấn luyện agent, và cuối cùng mới chuyển sang giai đoạn đánh giá.

Sau khi huấn luyện hoàn tất, hệ thống in ra số lượng trạng thái mà agent đã học được trong Q-table, từ đó làm đầu vào cho bước đánh giá ở phía sau.

6.2.3 Chạy đánh giá 2 chính sách trên 100 episode

Minh chứng:

```
# ═══════════════════════════════════════════════════════════
# BÀI 6 & 7: ĐÁNH GIÁ VÀ SO SÁNH (Q-LEARNING vs ROUND ROBIN)
# ═══════════════════════════════════════════════════════════
print("\n2. Đang tiến hành đánh giá 2 chiến lược trên 100 episode ... ")
eval_episodes = 100
eval_steps_per_episode = 100
eval_seed_base = 2026

# Hàm chạy 1 chiến lược và thu thập lịch sử
def evaluate_policy(policy_name, \
                    num_episodes=eval_episodes,
                    steps_per_episode=eval_steps_per_episode,
                    seed_base=eval_seed_base):
    history = []

    for episode in range(num_episodes):
        episode_seed = seed_base + episode
        obs, _ = env.reset(seed=episode_seed)
        if hasattr(env, 'rr_index'):
            env.rr_index = 0 # Reset lại bộ đếm Round Robin cho công bằng

        episode_reward = 0.0
        state = obs
        info = {}

        for step in range(steps_per_episode):
            if policy_name == "Q-learning":
                # Dùng trí khôn 100% (Greedy Action), không random nữa
                action = agent.choose_greedy_action(state)
            else:
                action = load_balancing_policy(option="round_robin", env=env)

            next_obs, reward, terminated, truncated, info = env.step(action)
            episode_reward += reward
            state = next_obs
```

Giải thích:

Trong phần này, sau khi đã huấn luyện xong agent, nhóm tiến hành xây dựng giai đoạn đánh giá để so sánh trực tiếp giữa 2 chính sách là Q-learning và Round Robin.

Nhóm thiết lập số episode đánh giá là 100, mỗi episode gồm 100 bước, đồng thời đặt một seed_base để đảm bảo việc sinh task trong quá trình đánh giá có tính ổn định và công bằng giữa hai chính sách.

Nhóm định nghĩa hàm evaluate_policy() để gom toàn bộ logic chạy thử một policy vào cùng một nơi. Trong mỗi episode, môi trường được reset lại với seed = seed_base + episode, nhờ đó episode thứ i của Q-learning và episode thứ i của Round Robin sẽ được kiểm tra trên cùng một điều kiện sinh task. Điều này giúp kết quả so sánh giữa hai policy có ý nghĩa hơn.

Nếu policy được chọn là Q-learning, agent sẽ sử dụng hoàn toàn hành động tốt nhất bằng hàm choose_greedy_action(state), tức là chỉ khai thác kiến thức đã học được trong Q-table mà không còn khám phá ngẫu nhiên nữa. Ngược lại, nếu policy là Round Robin, hệ thống sẽ gọi load_balancing_policy() để phân phối task tuần tự qua các server.

Ngoài ra, trước mỗi episode của Round Robin, nhóm còn đặt lại env.rr_index = 0 để bảo đảm bộ đếm Round Robin luôn được khởi tạo lại từ đầu. Việc này giúp các episode đánh giá độc lập với nhau và tránh việc kết quả bị ảnh hưởng bởi trạng thái còn sót lại từ episode trước.

Trong suốt quá trình chạy, hệ thống cộng dồn episode_reward và cập nhật trạng thái mới sau mỗi lần thực hiện env.step(action). Đây chính là phần lõi của giai đoạn đánh giá, làm cơ sở để thu thập các metrics ở bước tiếp theo.

6.2.4. Lưu metrics vào file .csv

Minh chứng:

```python
def save_metrics(history_ql, history_rr, metrics_dir="metrics"):
    """Convert evaluation histories to DataFrames and save strict CSV files in a metrics folder."""
    from pathlib import Path

    df_ql = pd.DataFrame(history_ql)
    df_rr = pd.DataFrame(history_rr)
    cwd = Path.cwd()
    if cwd.name == "notebooks":
        repo_root = cwd.parent
    elif (cwd / "notebooks").exists():
        repo_root = cwd
    else:
        repo_root = cwd
    metrics_path = repo_root / metrics_dir
    metrics_path.mkdir(parents=True, exist_ok=True)
    ql_file_path = metrics_path / "q_learning_metrics.csv"
    rr_file_path = metrics_path / "round_robin_metrics.csv"
    summary_file_path = metrics_path / "summary_metrics.csv"
    export_columns = [
        "Total_Runtime",
        "Total_Created_Tasks",
        "Accepted_Tasks",
        "Dropped_Tasks",
        "Completed_Tasks",
        "Reward"
    ]
    df_ql[export_columns].to_csv(ql_file_path, index=False)
    df_rr[export_columns].to_csv(rr_file_path, index=False)
    summary_ql = summarize_metrics(df_ql)
    summary_rr = summarize_metrics(df_rr)
    summary_df = pd.DataFrame([
        {"Metric": "Average Reward", "Q-learning": summary_ql["Average Reward"], "Round Robin": summary_rr["Average Reward"]},
        {"Metric": "Drop Rate", "Q-learning": summary_ql["Drop Rate"], "Round Robin": summary_rr["Drop Rate"]},
        {"Metric": "Completion Rate", "Q-learning": summary_ql["Completion Rate"], "Round Robin": summary_rr["Completion Rate"]}
    ])
    summary_df.to_csv(summary_file_path, index=False)
    print(f"Đã xuất dữ liệu Q-learning ra file '{ql_file_path.name}'!")
    print(f"Đã xuất dữ liệu Round Robin ra file '{rr_file_path.name}'!")
    print(f"Đã xuất bảng tổng hợp ra file '{summary_file_path.name}'!")
    return df_ql, df_rr, summary_df
```

Hình 33. Hàm save_metrics () lưu các thông số vào file .csv

Giải thích:

Sau khi đã thu thập được lịch sử chạy của hai chính sách thông qua history_ql và history_rr, nhóm xây dựng hàm save_metrics() để thực hiện việc lưu trữ dữ liệu ra file .csv. Hàm này trước hết chuyển lịch sử của từng policy thành DataFrame, sau đó xác định đúng thư mục gốc của project để tạo thư mục metrics nếu chưa tồn tại.

Tiếp theo, nhóm chỉ chọn đúng 6 cột metrics theo yêu cầu đề bài gồm:

- Total_Runtime
- Total_Created_Tasks
- Accepted_Tasks
- Dropped_Tasks
- Completed_Tasks
- Reward

Sau đó, hệ thống ghi lần lượt dữ liệu của chính sách Q-learning vào file q_learning_metrics.csv và dữ liệu của chính sách Round Robin vào file round_robin_metrics.csv. Việc tách thành 2 file riêng giúp dữ liệu rõ ràng, không bị dư cột Policy, đồng thời bám sát đúng yêu cầu lưu các metrics của từng chiến lược.

Ngoài ra, trong cùng hàm này, nhóm cũng tạo thêm file summary_metrics.csv để chuẩn bị cho bước tổng hợp và so sánh hiệu năng ở phần sau. Cuối cùng, hàm in ra tên các file đã xuất thành công và trả về các DataFrame để tiếp tục sử dụng cho việc tính toán bảng so sánh và vẽ biểu đồ.

6.2.5. Hàm tổng hợp metrics

Minh chứng:

```
def summarize_metrics(df):
    """Aggregate episode-level metrics for the comparison table."""
    total_created = df["Total_Created_Tasks"].sum()
    dropped = df["Dropped_Tasks"].sum()
    completed = df["Completed_Tasks"].sum()
    average_reward = df["Reward"].mean()
    drop_rate = (dropped / total_created) if total_created > 0 else 0.0
    completion_rate = (completed / total_created) if total_created > 0 else 0.0
    return {
        "Average Reward": average_reward,
        "Drop Rate": drop_rate,
        "Completion Rate": completion_rate
    }
```

Hình 34. Hàm tổng hợp metrics

Giải thích:

Sau khi đã lưu dữ liệu thô của từng policy vào các file .csv, nhóm xây dựng thêm hàm summarize_metrics(df) để tổng hợp các chỉ số quan trọng ở mức toàn bộ quá

trình đánh giá. Mục tiêu của hàm này là chuyển dữ liệu chi tiết theo từng episode thành các metrics tổng quát, phục vụ cho bước điền bảng so sánh ở phần tiếp theo.

Cụ thể, hàm sẽ tính tổng số task được tạo ra (Total_Created_Tasks), tổng số task bị drop (Dropped_Tasks) và tổng số task đã hoàn thành (Completed_Tasks) trên toàn bộ 100 episode. Đồng thời, hàm cũng tính Average Reward bằng cách lấy giá trị trung bình của cột Reward.

Từ các giá trị tổng này, nhóm tiếp tục suy ra hai chỉ số quan trọng là:

- Drop Rate = Dropped_Tasks / Total_Created_Tasks
- Completion Rate = Completed_Tasks / Total_Created_Tasks

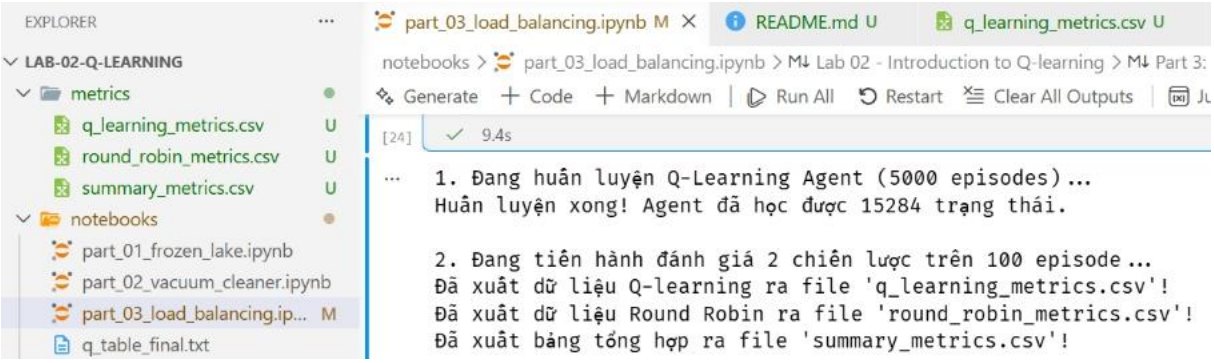
Ngoài ra, hàm còn kiểm tra trường hợp total_created = 0 để tránh lỗi chia cho 0, khi đó các tỉ lệ sẽ được gán bằng 0.0.

Kết quả cuối cùng được trả về dưới dạng dictionary gồm 3 metrics là Average Reward, Drop Rate và Completion Rate. Đây chính là các chỉ số được dùng để lập bảng so sánh giữa Q-learning và Round Robin trong phần sau của báo cáo.

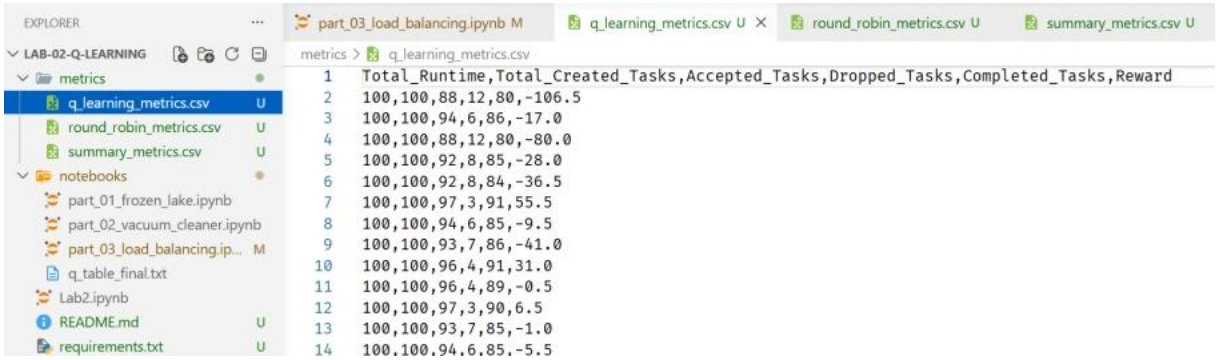
6.3. Kết quả

Sau khi hoàn tất quá trình huấn luyện, nhóm tiến hành chạy môi trường với hai chính sách là Q-learning và Round Robin trong 100 episode. Trong quá trình này, hệ thống thu thập và lưu trữ đầy đủ các metrics theo yêu cầu đề bài, bao gồm: tổng thời gian chạy, tổng số task được tạo ra, số task được tiếp nhận, số task bị drop, số task đã hoàn thành và reward.

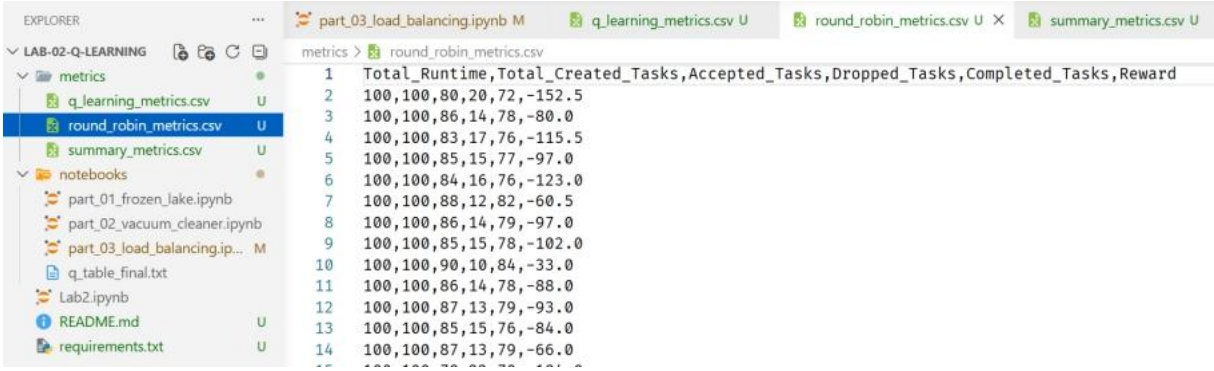
Minh chứng:



Hình 35. Kết quả sau khi chạy môi trường với hai chính sách.



Hình 36. File q_learning_metrics.csv lưu trữ các metrics của policy q-learning sau khi thực thi

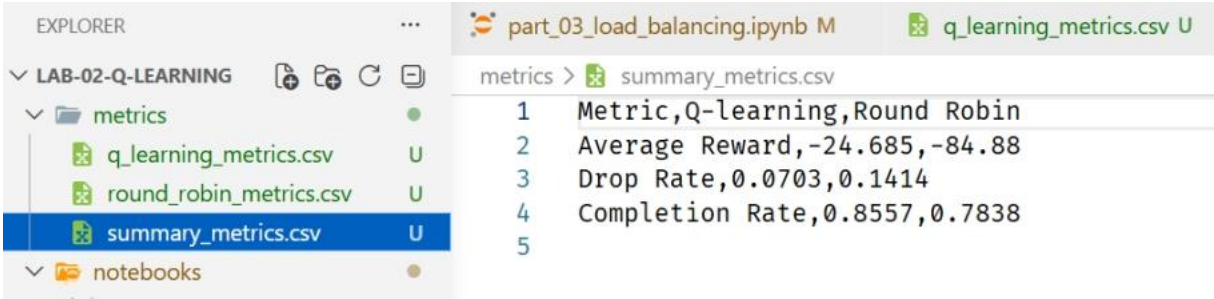


Hình 37. File round_robin_metrics.csv lưu trữ các metrics của policy round robin sau khi thực thi.

Giải thích:

Kết quả, chương trình sinh ra hai file dữ liệu chính trong thư mục metrics. Bao gồm q_learning_metrics.csv và round_robin_metrics.csv. Mỗi file lưu dữ liệu của một policy riêng biệt, với đúng 6 cột metrics theo yêu cầu đề bài. Đây là dữ liệu thô dùng làm cơ sở cho bước tổng hợp và so sánh hiệu năng ở phần tiếp theo.

Ngoài ra, nhóm cũng tạo thêm file summary_metrics.csv để phục vụ cho bước tính toán các chỉ số tổng hợp như Average Reward, Drop Rate và Completion Rate. File này được sử dụng ở câu tiếp theo để điền bảng so sánh và hỗ trợ trực quan hóa kết quả giữa hai chính sách.



Hình 38. File summary_metrics.csv lưu trữ các chỉ số tổng hợp

Câu hỏi 7: Từ các số liệu thu thập được, viết hàm tính để tổng hợp các metrics và điền vào bảng so sánh bên dưới. Vẽ biểu đồ thể hiện sự biến thiên của từng policy theo thời gian với 3 metrics: reward, drop, completion.

7.1. Ý tưởng

Từ các file dữ liệu thô đã thu thập được ở câu trước, nhóm tiếp tục xây dựng hàm tổng hợp metrics để rút ra các chỉ số đại diện cho toàn bộ quá trình đánh giá của từng policy. Mục tiêu của bước này là chuyển dữ liệu chi tiết theo từng episode thành các giá trị tổng quát hơn, từ đó dễ dàng điền vào bảng so sánh và trực quan hóa bằng biểu đồ.

Cụ thể, nhóm sử dụng các dữ liệu đã lưu của hai chính sách Q-learning và Round Robin để tính ra 3 chỉ số quan trọng nhất, gồm:

- Average Reward

- Drop Rate
- Completion Rate

Sau đó, nhóm dùng các chỉ số này để lập bảng so sánh hiệu năng giữa hai policy. Đồng thời, từ dữ liệu theo từng episode, nhóm vẽ 3 biểu đồ thể hiện sự biến thiên của reward, drop rate và completion rate theo thời gian, qua đó quan sát rõ xu hướng hoạt động của từng chiến lược.

7.2 Logic Code

7.2.1. Hàm tổng hợp metrics

Minh chứng:

```
def summarize_metrics(df):
    """Aggregate episode-level metrics for the comparison table."""
    total_created = df["Total_Created_Tasks"].sum()
    dropped = df["Dropped_Tasks"].sum()
    completed = df["Completed_Tasks"].sum()
    average_reward = df["Reward"].mean()
    drop_rate = (dropped / total_created) if total_created > 0 else 0.0
    completion_rate = (completed / total_created) if total_created > 0 else 0.0
    return {
        "Average Reward": average_reward,
        "Drop Rate": drop_rate,
        "Completion Rate": completion_rate
    }
```

Hình 39. Hàm tổng hợp metrics

Giải thích:

Nhóm xây dựng hàm summarize_metrics(df) để tổng hợp dữ liệu đánh giá của từng policy sau 100 episode. Hàm này nhận vào một DataFrame chứa các metrics thô của một chính sách, sau đó tính tổng số task được tạo ra, tổng số task bị drop, tổng số task đã hoàn thành và reward trung bình.

Từ các giá trị tổng này, nhóm tiếp tục suy ra hai chỉ số quan trọng là:

- Drop Rate = Dropped_Tasks / Total_Created_Tasks
- Completion Rate = Completed_Tasks / Total_Created_Tasks

Ngoài ra, hàm còn kiểm tra trường hợp total_created = 0 để tránh lỗi chia cho 0. Kết quả cuối cùng được trả về dưới dạng dictionary gồm 3 metrics là Average Reward, Drop Rate và Completion Rate. Đây chính là cơ sở để nhóm điền bảng so sánh giữa hai policy.

7.2.2. Chuẩn bị dữ liệu tổng hợp cho bảng so sánh

Minh chứng:

```python
# Chạy thu thập dữ liệu
history_ql = evaluate_policy("Q-learning")
history_rr = evaluate_policy("Round Robin")

# Xuất ra file CSV theo yêu cầu
df_ql, df_rr, summary_df = save_metrics(history_ql, history_rr)

summary_ql = summarize_metrics(df_ql)
summary_rr = summarize_metrics(df_rr)
```

Hình 40. Thu thập dữ liệu và xuất ra các file .csv

Giải thích:

Trước hết, nhóm gọi hàm evaluate_policy(...) cho hai chính sách Q-learning và Round Robin để thu thập dữ liệu đánh giá trên 100 episode. Kết quả trả về được lưu vào history_ql và history_rr.

Sau đó, nhóm sử dụng hàm save_metrics(history_ql, history_rr) để chuyển dữ liệu lịch sử thành các DataFrame, đồng thời lưu ra các file .csv phục vụ cho việc báo cáo và đối chiếu kết quả.

Từ hai DataFrame này, nhóm tiếp tục gọi hàm summarize_metrics(...) để rút ra 3 chỉ số tổng hợp quan trọng của từng policy là Average Reward, Drop Rate và Completion Rate.

Đây chính là dữ liệu đầu vào để nhóm điền bảng so sánh hiệu năng giữa hai chính sách ở bước tiếp theo.

7.2.3. Điền bảng so sánh hiệu năng

Minh chứng:

```python
# In Bảng So Sánh theo format của đề bài
print("\nBẢNG SO SÁNH HIỆU NĂNG TRÊN 100 EPISODE:")
print(f"{'Metric':<20} | {'Q-learning':<15} | {'Round Robin':<15}")
print("-" * 55)

avg_reward_ql = summary_ql['Average Reward']
avg_reward_rr = summary_rr['Average Reward']
print(f"{'Average Reward':<20} | {avg_reward_ql:<15.2f} | {avg_reward_rr:<15.2f}")

drop_rate_ql = summary_ql['Drop Rate']
drop_rate_rr = summary_rr['Drop Rate']
print(f"{'Drop Rate':<20} | {drop_rate_ql:<15.2%} | {drop_rate_rr:<15.2%}")

comp_rate_ql = summary_ql['Completion Rate']
comp_rate_rr = summary_rr['Completion Rate']
print(f"{'Completion Rate':<20} | {comp_rate_ql:<15.2%} | {comp_rate_rr:<15.2%}")

# Làm mượt biểu đồ để dễ quan sát xu hướng theo episode
smoothing_window = 10
plot_df_ql = df_ql.copy()
plot_df_rr = df_rr.copy()
for metric in ['Reward', 'Drop_Rate', 'Completion_Rate']:
    plot_df_ql[metric] = plot_df_ql[metric].rolling(window=smoothing_window, min_periods=1).mean()
    plot_df_rr[metric] = plot_df_rr[metric].rolling(window=smoothing_window, min_periods=1).mean()
```

Giải thích:

Ở bước tiếp theo, chương trình trích xuất các chỉ số tổng hợp từ summary_ql và summary_rr, sau đó in ra bảng so sánh theo đúng format của đề bài. Ba chỉ số được sử dụng để so sánh là:

- Average Reward
- Drop Rate
- Completion Rate

Nhờ bảng này, nhóm có thể quan sát trực tiếp hiệu năng tổng quát của hai chính sách trên toàn bộ 100 episode. Đây là cơ sở để đánh giá chính sách nào hoạt động hiệu quả hơn trên môi trường Load Balancing.

Ngoài ra, ở cuối block, nhóm cũng chuẩn bị dữ liệu đã làm mượt bằng moving average với window = 10 để phục vụ cho bước vẽ biểu đồ ở mục tiếp theo.

7.2.4. Vẽ biểu đồ thể hiện sự biến thiên của từng policy theo thời gian

Minh chứng:

```
print("\n3. Biểu đồ thể hiện sự biến thiên của từng policy theo thời gian với 3 metrics: reward, drop, completion")

# Vẽ 3 biểu đồ
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 5))

# Biểu đồ Reward
ax1.plot(plot_df_ql['Episode'], plot_df_ql['Reward'], label='Q-Learning', color='blue')
ax1.plot(plot_df_rr['Episode'], plot_df_rr['Reward'], label='Round Robin', color='orange')
ax1.set_title('Episode Reward (Moving Average)')
ax1.set_xlabel('Episode'); ax1.set_ylabel('Reward'); ax1.legend(); ax1.grid(True)

# Biểu đồ Drop Rate
ax2.plot(plot_df_ql['Episode'], plot_df_ql['Drop_Rate'], label='Q-Learning', color='blue')
ax2.plot(plot_df_rr['Episode'], plot_df_rr['Drop_Rate'], label='Round Robin', color='orange')
ax2.set_title('Task Drop Rate (Thấp hơn là tốt)')
ax2.set_xlabel('Episode'); ax2.set_ylabel('Drop Rate'); ax2.legend(); ax2.grid(True)

# Biểu đồ Completion Rate
ax3.plot(plot_df_ql['Episode'], plot_df_ql['Completion_Rate'], label='Q-Learning', color='blue')
ax3.plot(plot_df_rr['Episode'], plot_df_rr['Completion_Rate'], label='Round Robin', color='orange')
ax3.set_title('Task Completion Rate (Cao hơn là tốt)')
ax3.set_xlabel('Episode'); ax3.set_ylabel('Completion Rate'); ax3.legend(); ax3.grid(True)

plt.tight_layout()
plt.show()
```

Hình 41. Vẽ biểu đồ thể hiện sự biến thiên của từng policy theo thời gian.

Giải thích:

Sau khi đã có dữ liệu theo từng episode, nhóm tiến hành vẽ 3 biểu đồ để trực quan hóa sự biến thiên hiệu năng của hai chính sách Q-learning và Round Robin theo thời gian. Trước khi vẽ, hệ thống in ra một dòng thông báo để đánh dấu phần biểu đồ tương ứng với 3 metrics: reward, drop và completion.

Nhóm sử dụng matplotlib để tạo 3 biểu đồ đặt cạnh nhau trong cùng một figure, giúp việc quan sát và đối chiếu giữa hai policy trở nên thuận tiện hơn. Cụ thể:

- Biểu đồ thứ nhất thể hiện sự thay đổi của Reward theo từng episode
- Biểu đồ thứ hai thể hiện sự thay đổi của Drop Rate
- Biểu đồ thứ ba thể hiện sự thay đổi của Completion Rate

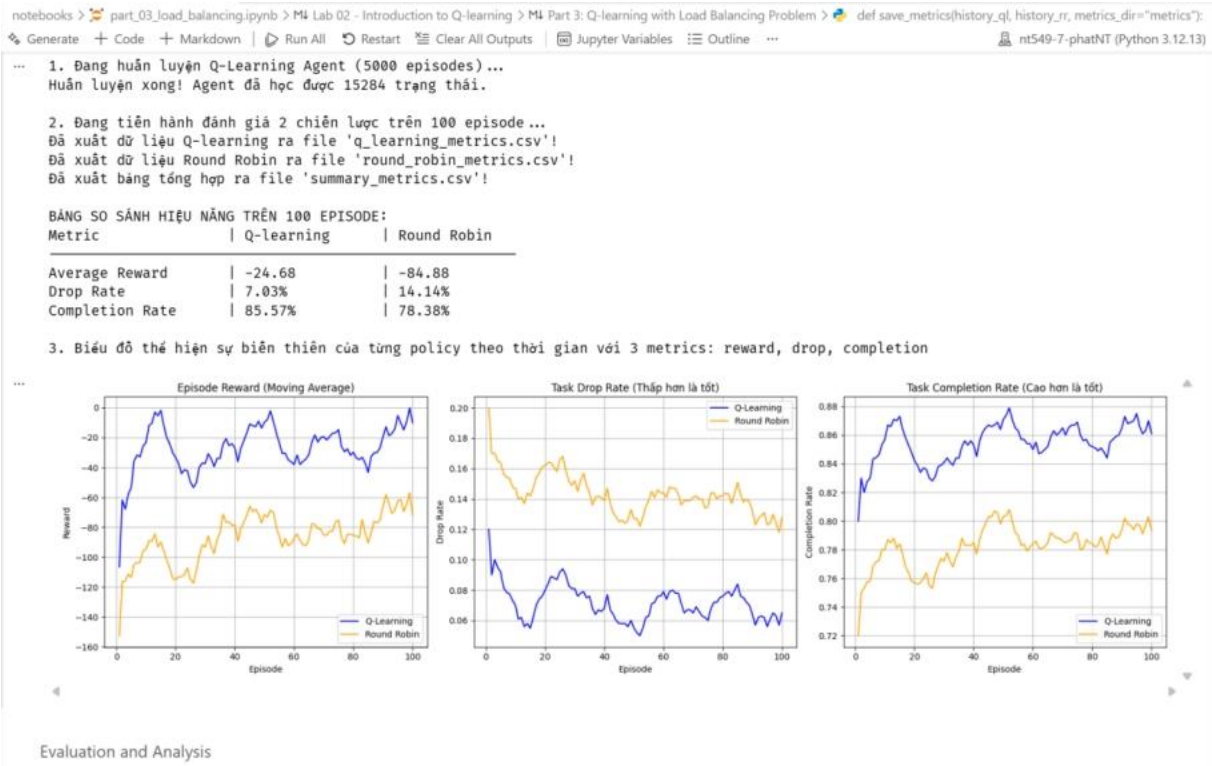
Trong mỗi biểu đồ, đường màu xanh biểu diễn chính sách Q-learning, còn đường màu cam biểu diễn chính sách Round Robin. Các trục đều được gắn nhãn rõ ràng, có tiêu đề riêng và thêm grid để người đọc dễ quan sát xu hướng hơn.

Nhờ ba biểu đồ này, nhóm có thể theo dõi trực quan sự khác biệt giữa hai chính sách trên toàn bộ quá trình đánh giá. Đây cũng là cơ sở quan trọng để đưa ra nhận xét về mức độ ổn định và hiệu quả của từng policy trong phần kết quả.

7.3 Kết quả thực thi

Sau khi chạy chương trình hoàn chỉnh, hệ thống lần lượt thực hiện các bước: huấn luyện Q-learning agent, đánh giá hai chính sách Q-learning và Round Robin trên 100 episode, lưu dữ liệu ra các file .csv, in bảng so sánh tổng hợp và hiển thị 3 biểu đồ trực quan.

Minh chứng:



Hình 42. Kết quả thực thi

Bảng so sánh:

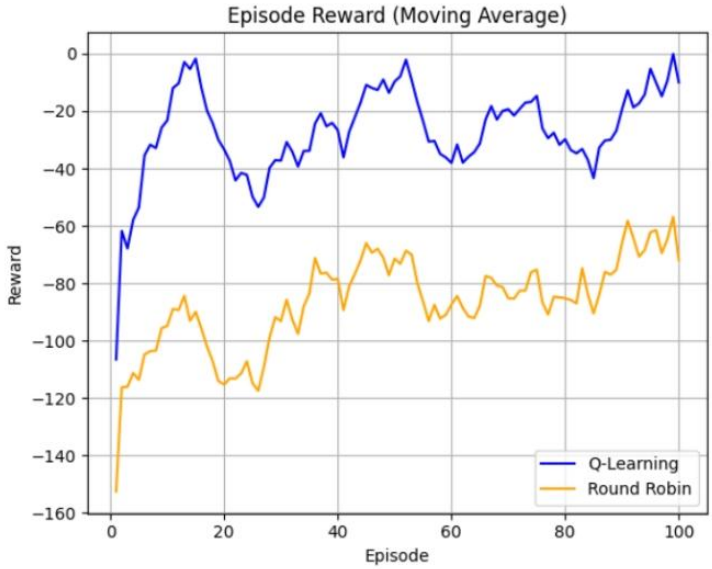
Metrics/ Giải thuật	Q-learning	Round Robin
Average Reward	-24.68	-84.88
Drop Rate	7.03%	14.14%
Completion Rate	85.57%	78.38%

Giải thích:

Biểu đồ thứ nhất thể hiện sự biến thiên của Reward theo từng episode. Có thể thấy đường màu xanh của chính sách Q-learning luôn nằm cao hơn rõ rệt so với đường màu cam của Round Robin trong gần như toàn bộ quá trình đánh giá.

Mặc dù cả hai đường đều dao động do đặc tính ngẫu nhiên của môi trường, reward của Q-learning ổn định hơn ở mức tốt hơn và có xu hướng tiến gần về 0, trong khi Round Robin luôn duy trì ở mức thấp hơn khá nhiều.

Điều này cho thấy chính sách học được từ Q-learning đã giúp hệ thống đưa ra quyết định phân phối task hiệu quả hơn, từ đó nhận được phần thưởng tổng thể cao hơn.

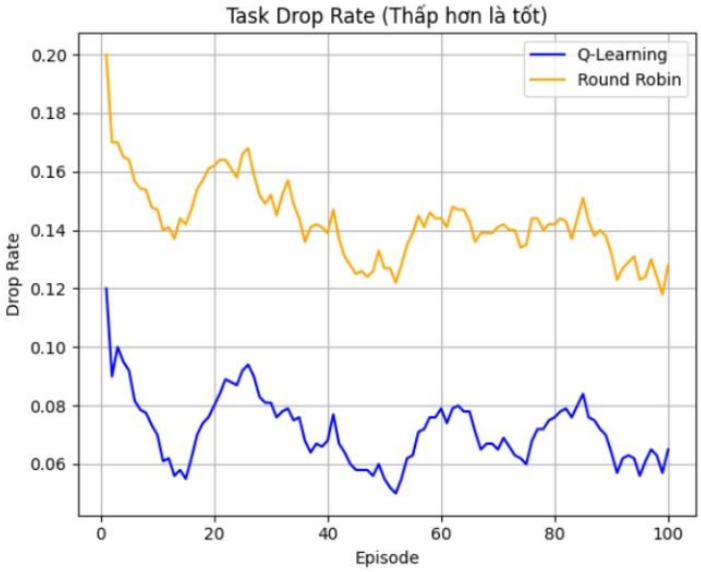


Hình 43. Biểu đồ thể hiện sự biến thiên của Reward theo từng Episode

Biểu đồ thứ hai thể hiện sự biến thiên của Drop Rate theo thời gian. Vì đây là chỉ số càng thấp càng tốt, đường màu xanh của Q-learning nằm thấp hơn đáng kể so với đường màu cam của Round Robin là một dấu hiệu tích cực.

Trong hầu hết các episode, Drop Rate của Q-learning chỉ dao động quanh mức thấp, trong khi Round Robin duy trì ở mức cao hơn tương đối ổn định.

Điều này cho thấy Q-learning đã phân phối tải hợp lý hơn, giảm hiện tượng dồn task vào các server đang quá tải, từ đó làm giảm số lượng task bị bỏ qua

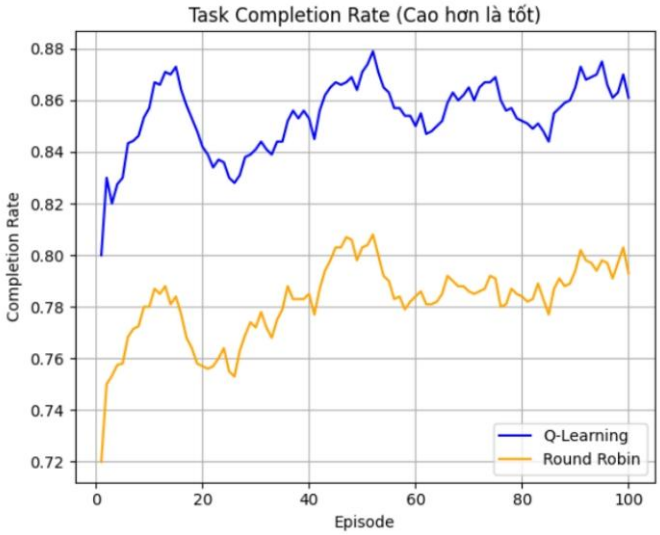


Hình 44. Biểu đồ thể hiện sự biến thiên của Drop Rate theo thời gian

Biểu đồ thứ ba thể hiện sự biến thiên của Completion Rate theo từng episode. Trái ngược với Drop Rate, đây là chỉ số càng cao càng tốt. Có thể thấy đường màu xanh

của Q-learning luôn nằm trên đường màu cam của Round Robin trong gần như toàn bộ quá trình đánh giá.

Completion Rate của Q-learning duy trì quanh mức cao hơn và tương đối ổn định, cho thấy số lượng task được xử lý hoàn tất lớn hơn đáng kể. Điều này chứng minh rằng Q-learning không chỉ giảm số task bị drop mà còn tận dụng tài nguyên hệ thống tốt hơn để hoàn thành nhiều công việc hơn.



Hình 45. Biểu đồ thể hiện sự biến thiên của Completion Rate theo từng Episode

Tổng hợp từ cả ba biểu đồ, có thể thấy chính sách Q-learning đều chiếm ưu thế hơn so với Round Robin ở cả ba khía cạnh: reward cao hơn, drop rate thấp hơn và completion rate cao hơn. Kết quả này phù hợp với bảng so sánh tổng hợp đã trình bày ở trên và cho thấy Q-learning là chính sách điều phối hiệu quả hơn trên môi trường Load Balancing đã xây dựng.

7.4 Nhận xét

Thông qua bảng so sánh và ba biểu đồ đã trình bày, có thể thấy chính sách Q-learning hoạt động hiệu quả hơn rõ rệt so với chính sách Round Robin trên môi trường Load Balancing. Cụ thể, Q-learning đạt reward trung bình cao hơn, tỉ lệ drop thấp hơn và tỉ lệ hoàn thành task cao hơn trong toàn bộ quá trình đánh giá.

Kết quả này cho thấy Q-learning đã học được cách phân phối task hợp lý hơn giữa các server, tránh tình trạng một số server bị quá tải trong khi các server khác chưa được khai thác hết. Nhờ đó, hệ thống giảm được số lượng task bị bỏ qua và đồng thời tăng số lượng task được xử lý hoàn tất.

Trong khi đó, chính sách Round Robin tuy đơn giản, dễ cài đặt và có tính công bằng theo thứ tự phân phối, nhưng lại không xét đến trạng thái thực tế của từng server tại thời điểm ra quyết định. Vì vậy, chính sách này dễ dẫn đến việc tiếp tục phân task vào các server đang bận hoặc gần đầy hàng đợi, làm tăng Drop Rate và giảm Completion Rate.

Từ các kết quả thực nghiệm, có thể kết luận rằng việc áp dụng Q-learning vào bài toán Load Balancing mang lại hiệu quả tốt hơn so với chiến lược phân phối tuần tự truyền thống. Điều này cho thấy học tăng cường là một hướng tiếp cận phù hợp đối với các bài toán điều phối tài nguyên động trong hệ thống mạng.

YÊU CẦU CHUNG

1) Đánh giá

- Chuẩn bị tốt các yêu cầu đặt ra trong bài thực hành.
- Sinh viên hiểu và tự thực hiện được bài thực hành, trả lời đầy đủ các yêu cầu đặt ra.
- Nộp báo cáo kết quả chi tiết những đã thực hiện, quan sát thấy và kèm ảnh chụp màn hình kết quả *(nếu có)*; giải thích cho quan sát *(nếu có)*.
- Sinh viên báo cáo kết quả thực hiện và nộp bài.

2) Báo cáo

- Nộp file .docx. Tập trung vào nội dung, giải thích.
- Nội dung trình bày bằng Font chữ Cambria hoặc Times New Roman (*tuy nhiên, phải chuyển đổi hết báo cáo này sang 1 font chữ thống nhất*) – cỡ chữ 13. Canh đều (Justify) cho văn bản. Canh giữa (Center) cho ảnh chụp.
- Đặt tên theo định dạng: STTNhom-LabX_MSSV1_MSSV2. (trong đó X là Thứ tự buổi Thực hành).

  Ví dụ: TH1.01-Lab1_25520001_25520002
- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Không đặt tên đúng định dạng – yêu cầu, sẽ KHÔNG chấm điểm bài thực hành.
- Nộp file báo cáo trên theo thời gian đã thống nhất tại courses.uit.edu.vn.

Bài sao chép, trễ, … sẽ được xử lý tùy mức độ vi phạm.

HẾT